

# 圆柱形药材热风干燥的径向传热传质与收缩域数值模拟

## 摘要

围绕圆柱形药材热风干燥的内部温湿场与停止时间，建立含水率相关物性的径向传热传质模型。采用环形有限体积与后向欧拉离散，以场增量和非线性残差共同控制 Picard 迭代；通过半单元传导和表面对流串联阻力重构真实表面值，并以未舍入单元最大含水率低于 0.15 kg/kg 检测干燥终点。

问题一使用常热物性及非线性扩散关系。1800 s 时，中心与表面温度分别为 33.5759 °C、36.7859 °C，含水率分别为 2.5500 kg/kg、1.5103 kg/kg，截面加权脱水比例为 0.100563。短期失水主要集中于外层，温升不能代替全域水分达标判断。

问题二从初态统一采用附录 3 物性。3 h 时中心与表面温度分别为 49.8495 °C、49.9664 °C，含水率分别为 1.7662 kg/kg、1.0081 kg/kg。温差缩小后仍存在明显含水率梯度，完整秒级输出持续至问题三终点。

问题三保持 2 cm 半径，得到达标节点 205818 s，即 57.1717 h，时间夹逼为 205817.5—205818 s。终点中心、表面含水率显示为 0.1500 kg/kg、0.0525 kg/kg，中心显示值等于阈值源于舍入。三个独立边界情景的时长相对变化为 -0.3197%、+0.4628% 和 +0.5425%，仅表示所测边界假设的影响。

问题四在均匀径向收缩假设下引入附件 2 半径和附录 4 物性，得到 182969.5 s，即 50.8249 h，较问题三缩短 11.1013%，终点半径 1.2 cm。增加附录 4 物性与固定半径的中间对照得到 129.0943 h，表明物性和几何沿指定路径产生相反作用；三格对照不能识别交互。

正式计算采用 1280 单元、0.5 s。相较 2560 单元、0.25 s 参考，两问最大含水率差均低于 0.0005 kg/kg，时长相对差不超过 0.0027%；累计归一水分收支偏差接近浮点精度。加密与收支支持离散可靠性，但缺少内部场独立观测，预测的实验准确度仍需验证。

关键词：药材烘干；干基含水率；有限体积法；阈值检测；径向收缩

# 1 问题重述

## 1.1 研究背景与工程意义

药材干燥的停止决策取决于内部水分，而温度与表面状态通常更容易观察。热风先加热外层，水分又必须通过内部迁移才能到达表面；这种空间差异使“烘房达到设定温度”“表面变干”和“全部位置达标”不能互相替代。本文围绕题给尺寸、物性和边界数据，计算径向场及其首次达标节点，重点区分温升时间尺度与脱水时间尺度。多相干燥研究提示真实过程还涉及潜热、孔隙和材料结构变化 [3-5]，但题目缺少相应参数，因此不把当前宏观模型扩展成未经标定的品质或能耗预测模型。

研究对象长度为 25 cm、初始半径为 2 cm，初始温度为 28 °C，初始干基含水率为 2.55 kg/kg。附件 1 给出烘房空气的温度和水分浓度，附件 2 给出药材半径随时间的变化。空气水分浓度的单位是 kg/kg，不能直接替换成相对湿度百分数。物性关系中的温度使用 K，展示计算结果时再换算成 °C。本文在后续推导中统一用“含水率”指药材的干基含水率，空气变量则明确称为“空气水分浓度” [6]。

## 1.2 问题提出

问题一研究 0—1800 s 的预热阶段。密度、比热容和导热系数分别取 820 kg/m<sup>3</sup>、2600 J/(kg·K)、0.36 W/(m·K)，扩散系数随含水率非线性变化。需给出指定七个时刻、五个径向位置的温度与含水率，并保存每秒、每隔 0.1 cm 的完整分布。

问题二要求从初始时刻统一采用题面附录 3 的经验物性，建立覆盖整个干燥过程的模型。正文表 3、表 4 只展示前 3 h、每隔 0.5 h 的结果，完整附件则应持续记录至干燥结束。因此展示窗口与求解终点应分别设定，不能把表格仅要求前 3 h 误解为全过程只需计算 3 h。

问题三在固定半径下确定全域含水率均低于 0.15 kg/kg 的首次时刻，并列每隔 6 h 及终点的径向分布。问题四进一步引入实测半径变化和附录 4 物性，给出对应终点、固定物理位置和真实表面处的含水率。若某一物理位置已位于收缩后的药材外部，应标记为不适用，不能将其填成零含水率。四个问题所需完整附件的命名、时间步距和数据路径列于附录。

## 1.3 任务之间的关系

四问宜理解为三个初边值问题、四类输出任务。常热物性的预热算例独立回答问题一；附录 3 模型从同一初态出发，其前 3 h 分布回答问题二，长期阈值事件回答问题三；附录 4 模型再配合实测半径回答问题四。它们共用离散框架而不拼接末态。下文先给出共同假设、输入与符号，再推导固定域和物质坐标域的方程，最后比较数值设置与边界情景。这样可以沿“模型条件—计算轨迹—输出位置—判定规则”追溯每个结论，避免将物性差异误解为阶段衔接误差。

## 2 问题分析

### 2.1 空间尺度与耦合关系

长径比  $L/(2R)=6.25$  支持将中段作为径向近似的研究对象，但不足以证明端部效应消失。本文保留圆柱几何，将径向区间划分为环形控制体；相邻控制体共享通量，使内部交换在全域求和时成对抵消 [1, 2]。温度改变扩散系数，含水率又改变热容量与导热系数，因此两个场应在同一时间步内更新。与之相比，先独立算温度再永久冻结温度输入，会丢失题给物性的动态耦合，不能代表本题全过程模型。

### 2.2 预热模型的分析重点

预热算例能够辨别两种响应：常热物性下的温度传播，以及含水率控制的非线性扩散。均匀初场并不意味着后续均匀，表面对流交换首先产生外层梯度，再向内传播。扩散系数随位置变化，必须保留在散度内部；只取局部系数乘拉普拉斯会遗漏系数梯度的作用。输出还需区分单元中心、对称轴及真实表面，尤其不能把最外层单元中心当作表面测点。短时加密用于检验这一算例，长期可靠性则另外由低含水率阶段的场比较确认。

### 2.3 全过程经验物性的处理

问题二的密度、比热容和导热系数随含水率变化，扩散系数同时受含水率与绝对温度影响。局部系数与未知场相互依赖，因此每个时间步用上一时刻的场初始化，通过 Picard 迭代更新系数，求解含水率与温度三对角系统。接受时间步前同时检查场增量和按最终物性重组的方程残差；未收敛时报告失败，不通过剪裁含水率伪装为可接受解。这样才能把非线性迭代误差与空间、时间离散误差分开讨论。

### 2.4 长期终点与环境延拓

问题三的关键在于“全域均达标”，因此采用所有径向单元的含水率最大值作为停止依据，同时记录相邻时间步的最大值及中心重构偏差。环境数据仅覆盖 4 h，长期积分需要明确外推假设。基准保持末值；对照采用最后一小时按时间积分得到的平均环境，并将等效空气含水率分别乘以 0.8 和 1.2。各情景独立检测终点，不能把达到相同运行上限误当作得到相同干燥时间。这些确定性情景也不构成统计意义上的置信区间。

### 2.5 收缩域的分析与比较边界

问题四把空间坐标写成当前半径的比例，以跟踪相同的相对径向位置。在均匀径向收缩假设下，材料运动与网格运动一致，含水率按单位干物质质量定义，物质坐标内的扩散项由当前半径平方进行缩放。该近似与静止介质中仅移动计算边界的模型不同，推导时必须先声明材料运动假

设，再进行坐标变换。恒半径退化试验能检验实现的一致性，但无法单独证明真实收缩过程满足该假设。

由于问题四同时更换物性，直接比较两问只能得到综合时间差。本文补充“附录4物性、固定半径”这一中间情景，形成先改变物性、再改变几何的三格顺序对照。沿指定路径能够计算两步时长差，但缺少“附录3物性、收缩半径”组合，不能进一步识别交互作用或给出与路径无关的独立贡献率。把比较对象先定义清楚，能够避免由较短干燥时间直接推出某个机制占据全部作用。

## 3 模型假设

### 3.1 模型基本假设

(1) 药材近似为长圆柱体，中段截面的场轴对称，忽略轴向温度与含水率梯度。外部环境在周向上均匀，中心满足对称边界。该假设将空间问题降为一维，对靠近两端的区域不直接给出同等精度承诺。

(2) 将内部多相水分迁移等效为宏观扩散，局部系数采用题面经验式。模型没有单独追踪液水、蒸汽和束缚水，也没有显式加入蒸发潜热、辐射及化学反应项。因此温度场是给定等效物性下的导热近似，不代表完整的多相能量平衡 [3, 5]。

(3) 外表面的热量与水分通量均采用 Robin 形式。温差直接驱动换热；水分通量以药材干基含水率与空气水分浓度之差作为等效驱动力。两种浓度的质量基准不同，题面没有提供吸附等温线或相平衡换算，本文据给定传质系数采用此工程闭合关系，并将其列为模型误差来源。

(4) 附件1相邻采样点之间线性插值，超过 14400 s 后基准保持末值；环境情景另行说明。附件2采用分段线性插值，问题四在其 72 h 观测范围内结束，不需外推终点半径。129.0943 h 的中间对照采用固定半径，不使用收缩曲线的远期外推。原始半径允许轻微起伏，未人为改成严格单调曲线。

(5) 问题四假定长度不变、径向均匀收缩且无干物质损失，材料点速度与径向位置成正比。相对半径标记同一材料层，干骨架密度随截面积缩小而增加。干骨架密度用于解释干物质守恒，与题给热容量关系中的经验密度分别定义，不能将两者强行视为同一物理量。

(6) 初始场均匀；内部浮点运算不作四位小数舍入，阈值检测使用未舍入含水率。正文和电子表格的展示精度按题面要求处理。输出精度、时间离散精度和物理预测误差是不同概念。

### 3.2 假设的验证边界

环境和半径记录是模型输入，不是独立的内部温湿场验证样本。现有数据足以运行上述确定性模型并进行数值自洽性检查，但不足以估计全部物性参数的不确定分布，也不足以给出实测误差置信区间。后续结论均限定于所声明的边界、物性和几何假设。

# 4 符号说明

## 4.1 主要数学符号与物理量定义

为便于下文建立数学模型与推导离散格式，将本文中使用的符号、物理意义及其 SI 单位汇总于表 4-1 中。

表 4-1 主要数学符号与物理量定义

| 符号              | 物理意义                             | 单位                |
|-----------------|----------------------------------|-------------------|
| $r$             | 径向空间坐标（距圆柱中心轴距离）                 | $m$               |
| $t$             | 干燥时间坐标                           | $s$               |
| $R(t)$          | 药材圆柱的外表面半径                       | $m$               |
| $L$             | 药材圆柱的长度                          | $m$               |
| $T(r, t)$       | 药材在位置 $r$ 、时刻 $t$ 处的温度           | $^{\circ}C$ 或 $K$ |
| $C(r, t)$       | 药材在位置 $r$ 、时刻 $t$ 处的干基水分浓度（含水率）  | $kg/kg$           |
| $T_{air}(t)$    | 烘房内干燥热风环境的温度                     | $^{\circ}C$ 或 $K$ |
| $C_{air}(t)$    | 烘房内干燥热风环境的水分浓度                   | $kg/kg$           |
| $\rho$          | 题给经验密度，仅用于体积热容量                  | $kg/m^3$          |
| $\rho_d$        | 均匀收缩下的干骨架质量密度                    | $kg/m^3$          |
| $c_p$           | 药材的比定压热容                         | $J/(kg \cdot K)$  |
| $k$             | 药材的宏观有效导热系数                      | $W/(m \cdot K)$   |
| $D(C, T)$       | 水分在药材内部的非线性有效扩散系数                | $m^2/s$           |
| $h$             | 药材外表面的对流换热系数                     | $W/(m^2 \cdot K)$ |
| $h_m$           | 药材外表面的对流传质系数                     | $m/s$             |
| $\xi$           | 物质坐标系下的无量纲径向位置（ $\xi = r/R(t)$ ） | 无量纲               |
| $N$             | 径向离散有限控制容积（单元）数量                 | 无量纲               |
| $\Delta t$      | 数值计算的时间步长                        | $s$               |
| $C_{threshold}$ | 烘干达标判定临界干基含水率（0.1500）            | $kg/kg$           |

## 4.2 实验数据统计与特征分析

附件 1 共 241 条记录，间隔 60 s，覆盖 0—14400 s；温度和水分浓度没有缺失值。表 4-2 汇总其范围与平均水平，作为后续边界输入的基本核对。

表 4-2 烘房环境测试数据统计概览

| 变量     | 单位    | 样本数 | 缺失数 | 均值       | 标准差      | 最小值      | 最大值       |
|--------|-------|-----|-----|----------|----------|----------|-----------|
| 时间     | s     | 241 | 0   | 7200.000 | 4174.207 | 0.000    | 14400.000 |
| 空气温度   | °C    | 241 | 0   | 47.2453  | 5.1041   | 28.0000  | 50.2460   |
| 空气水分浓度 | kg/kg | 241 | 0   | 0.044756 | 0.008120 | 0.019630 | 0.050250  |

表 4-2 中的均值是观测样本统计量，不用于替换基准中的时间变化。相邻观测线性插值得到每个积分时刻的外部状态。最后一小时均值情景则对 10800—14400 s 的分段线性序列积分后除以 3600 s，不能用一组不等权端点的简单平均冒充时间平均。图 4-1 展示环境升温及水分浓度变化。

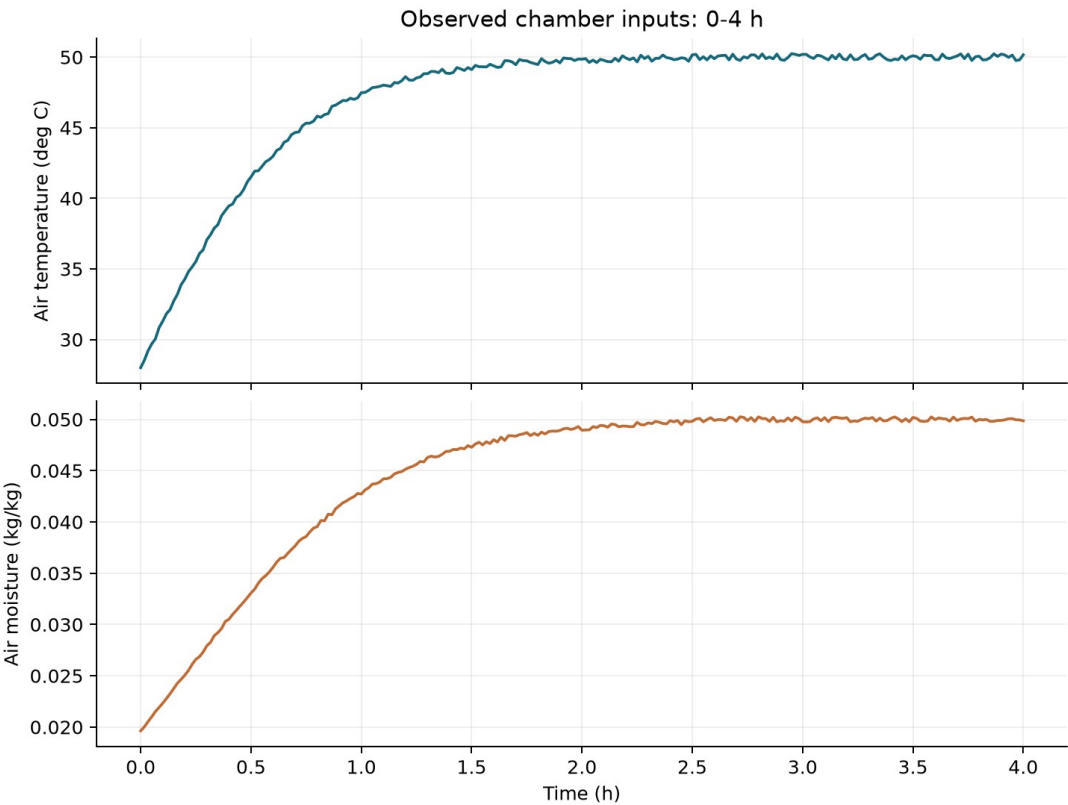


图 4-1 烘房空气温度与水分浓度时间序列

图 4-1 表明升温阶段与后期平台具有不同的驱动力。平台期的外界温度趋于稳定，并不意味着药材内部含水率同步达到稳定，因此需要在观测窗口结束后继续积分。延拓方案的差异在第 6 章单独讨论。

附件 2 共 145 条半径记录，间隔 1800 s，覆盖 0—259200 s。表 4-3 列出半径统计量，图 4-2 给出其变化历程。

表 4-3 药材实测半径收缩数据统计概览

| 变量 | 单位 | 样本数 | 缺失数 | 均值         | 标准差       | 最小值      | 最大值        |
|----|----|-----|-----|------------|-----------|----------|------------|
| 时间 | s  | 145 | 0   | 129600.000 | 75342.418 | 0.000    | 259200.000 |
| 半径 | cm | 145 | 0   | 1.246021   | 0.126608  | 1.198000 | 2.000000   |

表 4-3 中的最小半径是整个观测区间的极值，而达标时刻半径要按当时的插值结果单独计算，二者不能互相替代。

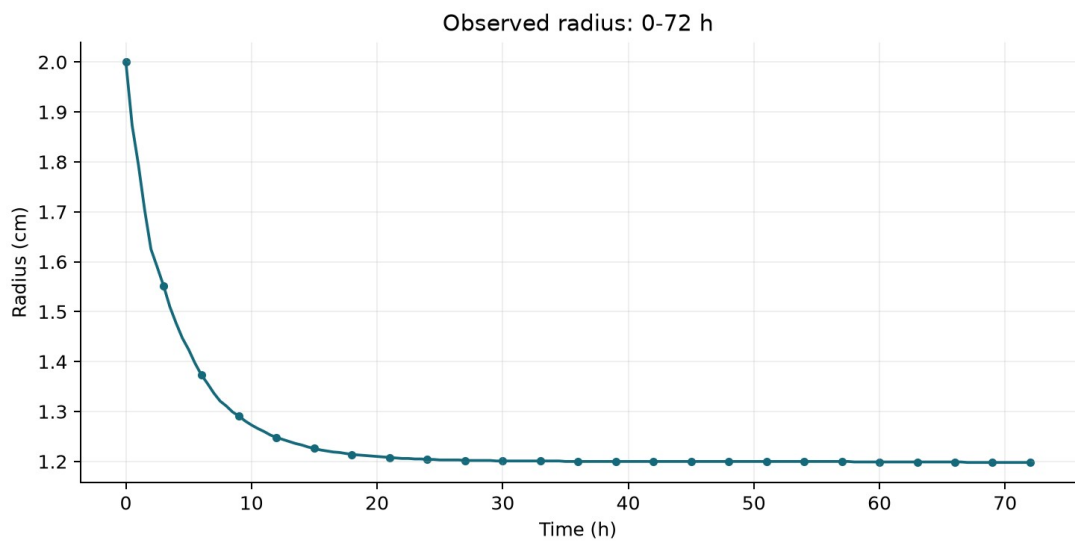


图 4-2 药材半径随烘干时间的变化

由图 4-2 可见，半径从初始 2 cm 降至约 1.2 cm 的后期水平。径向距离变化直接改变扩散项的几何尺度，但它对时间的实际影响还取决于物性和边界阻力，不能只用半径比直接换算完整干燥时间。

## 5 模型的建立与求解

### 5.1 问题一：预热阶段模型与求解

#### 5.1.1 建模思路

在预热平衡阶段（0-1800 s），初始冷态中药材置于升温的烘房环境中。烘房热风通过强迫对流向药材表面供热，驱动药材升温；同时表面水分蒸发形成浓度差，驱动内部水分向外扩散 [1, 2]。针对长圆柱体药材，忽略端部轴向耗散，在圆柱坐标系下建立一维非定常径向热传导与非线性水分扩散耦合偏微分方程组。

### 5.1.2 变量定义与公式推导

药材干基质量保持守恒，圆柱半径恒为  $R=0.02\text{ m}$ ，长度  $L=0.25\text{ m}$ 。控制方程包括温度场与水分浓度场的偏微分方程：

1. 导热微分方程：

$$\rho c_p \frac{\partial T}{\partial t} = \frac{1}{r} \frac{\partial}{\partial r} \left( r k \frac{\partial T}{\partial r} \right), \quad 0 < r < R, t > 0$$

其中， $\rho=820\text{ kg/m}^3$  为题面给定密度， $c_p=2600\text{ J/(kg}\cdot\text{K)}$  为比热容， $k=0.36\text{ W/(m}\cdot\text{K)}$  为导热系数。

2. 非线性水分扩散方程：

$$\frac{\partial C}{\partial t} = \frac{1}{r} \frac{\partial}{\partial r} \left( r D(C) \frac{\partial C}{\partial r} \right), \quad 0 < r < R, t > 0$$

其中，非线性有效水分扩散系数满足：

$$D(C) = 7 \times 10^{-9} \exp\left(-\frac{0.89}{C}\right) [\text{m}^2/\text{s}]$$

3. 定解条件（初始条件与边界条件）：

• 初始条件（ $t=0$ ）：

$$T(r, 0) = T_0 = 28.0000^\circ\text{C} = 301.15\text{ K}, \quad C(r, 0) = C_0 = 2.5500\text{ kg/kg}$$

• 中心对称边界条件（ $r=0$ ）：

$$\left. \frac{\partial T}{\partial r} \right|_{r=0} = 0, \quad \left. \frac{\partial C}{\partial r} \right|_{r=0} = 0$$

• 外表面 Robin 对流边界条件（ $r=R$ ）：

$$\begin{aligned} -k \left. \frac{\partial T}{\partial r} \right|_{r=R} &= h(T(R, t) - T_{air}(t)) \\ -D(C) \left. \frac{\partial C}{\partial r} \right|_{r=R} &= h_m(C(R, t) - C_{air}(t)) \end{aligned}$$

其中， $h=25\text{ W/(m}^2\cdot\text{K)}$  为表面对流换热系数， $h_m=8 \times 10^{-7}\text{ m/s}$  为对流传质系数， $T_{air}(t)$  与  $C_{air}(t)$  来自附件 1 环境时间序列。

### 5.1.3 求解算法

Step 1：将半径区间划分为  $N=1280$  个等宽圆柱壳层，时间步长为  $\Delta t=0.5\text{ s}$ 。该配置由长期场加密比较选定，四问统一使用。单元未知量位于壳层中心。设两侧界面半径为  $r_{i-1/2}$  与  $r_{i+1/2}$ ，单位长度体积和界面面积分别为：



$$V_i = \pi(r_{i+1/2}^2 - r_{i-1/2}^2), \quad A_{i+1/2} = 2\pi r_{i+1/2}$$

中心界面面积为零，因而对称条件可自然进入离散式，不需要直接计算原方程中表面上奇异的  $1/r$  项。

Step 2: 使用相邻单元物性的调和平均构造公共界面扩散通量。记界面传导能力为  $G_{i+1/2} = A_{i+1/2} D_{i+1/2} / \Delta r$ ，内点含水率的后向欧拉式为：

$$-G_{i-1/2} C_{i-1}^{n+1} + (V_i / \Delta t + G_{i-1/2} + G_{i+1/2}) C_i^{n+1} - G_{i+1/2} C_{i+1}^{n+1} = V_i C_i^n / \Delta t$$

温度方程采用同一结构，将储存项换成局部体积热容，将扩散能力换成导热能力。界面两侧使用相同通量、相反符号，避免相邻单元之间产生人为源项 [1]。

Step 3: 在外层单元中心到表面之间加入半单元传导阻力，再与表面对流阻力串联。含水率的等效边界系数为：

$$\gamma_C = \left( \frac{1}{h_m} + \frac{\Delta r}{2 D_N} \right)^{-1}, \quad \gamma_T = \left( \frac{1}{h} + \frac{\Delta r}{2 k_N} \right)^{-1}$$

这使外层未知量位于单元中心时仍能正确组装 Robin 边界。温度与含水率分别使用自己的边界阻力，不能互换系数。

Step 4: 每个时间步以旧场初始化，按当前迭代场组装两套三对角系统，分别求解含水率与温度，再同步更新迭代场。含水率最大增量不超过  $10^{-10}$  kg/kg、温度最大增量不超过  $10^{-7}$  K，且最终物性重组后的两个归一残差均不超过  $10^{-10}$ ，才接受该步。最多 50 轮是失败保护上限，不是每步固定迭代次数。若达到上限仍不满足条件，或者出现非有限场、非正含水率，则终止并报告失败，不剪裁解。三对角消元的复杂度随单元数线性增长 [1]。

Step 5: 保存时刻按物理时间控制，与积分步数分开计数。内点对单元中心值作线性插值；中心使用满足零梯度的局部二次重构。记第一、第二单元值为  $C_1, C_2$ ，则中心重构为  $C(0) \approx C_1 + (C_1 - C_2)/8$ ，温度同理。表面通过实际外层半单元阻力计算，而不是把最外层中心当作真实表面。令  $\delta = \Delta r/2$ ，有：

$$C_s = \frac{D_N C_N + h_m \delta C_{air}}{D_N + h_m \delta}, \quad T_s = \frac{k_N T_N + h \delta T_{air}}{k_N + h \delta}$$

该重构满足与边界离散一致的通量关系；即使单元中心场差别很小，端点处理方式仍可能显著改变表面值。因此正文表面列与图中的单元剖面必须按各自的坐标含义读取，不能用插值端值替代边界解。

5.1.4 结果分析

数值求解得出了 0-1800 s 内药材径向温湿场演变规律。表 5-1 和表 5-2 分别列出了在 100 s、300 s、600 s、900 s、1200 s、1500 s、1800 s 各指定时刻与径向位置（0 cm、0.5 cm、1.0 cm、1.5 cm、2.0 cm）处的温度与水分浓度计算数值。

表 5-1 30 分钟内药材的温度（表 1，单位：℃）

| 时间/s | 0 cm    | 0.5 cm  | 1 cm    | 1.5 cm  | 2 cm    |
|------|---------|---------|---------|---------|---------|
| 100  | 28.0001 | 28.0004 | 28.0042 | 28.0330 | 28.1804 |
| 300  | 28.0412 | 28.0639 | 28.1518 | 28.3686 | 28.8493 |
| 600  | 28.4541 | 28.5368 | 28.8047 | 29.3165 | 30.1657 |
| 900  | 29.3253 | 29.4592 | 29.8763 | 30.6168 | 31.7309 |
| 1200 | 30.5436 | 30.7107 | 31.2231 | 32.1132 | 33.4281 |
| 1500 | 31.9965 | 32.1875 | 32.7667 | 33.7468 | 35.1206 |
| 1800 | 33.5759 | 33.7726 | 34.3647 | 35.3626 | 36.7859 |

表 5-2 30 分钟内药材的水分浓度（表 2，单位：kg/kg）

| 时间/s | 0 cm   | 0.5 cm | 1 cm   | 1.5 cm | 2 cm   |
|------|--------|--------|--------|--------|--------|
| 100  | 2.5500 | 2.5500 | 2.5500 | 2.5500 | 2.2472 |
| 300  | 2.5500 | 2.5500 | 2.5500 | 2.5492 | 2.0510 |
| 600  | 2.5500 | 2.5500 | 2.5500 | 2.5352 | 1.8770 |
| 900  | 2.5500 | 2.5500 | 2.5497 | 2.5045 | 1.7548 |
| 1200 | 2.5500 | 2.5500 | 2.5482 | 2.4646 | 1.6586 |
| 1500 | 2.5500 | 2.5499 | 2.5445 | 2.4206 | 1.5788 |
| 1800 | 2.5500 | 2.5497 | 2.5383 | 2.3755 | 1.5103 |

图 5-1 展示了预热阶段温度与水分浓度的时空分布与径向演化特征。

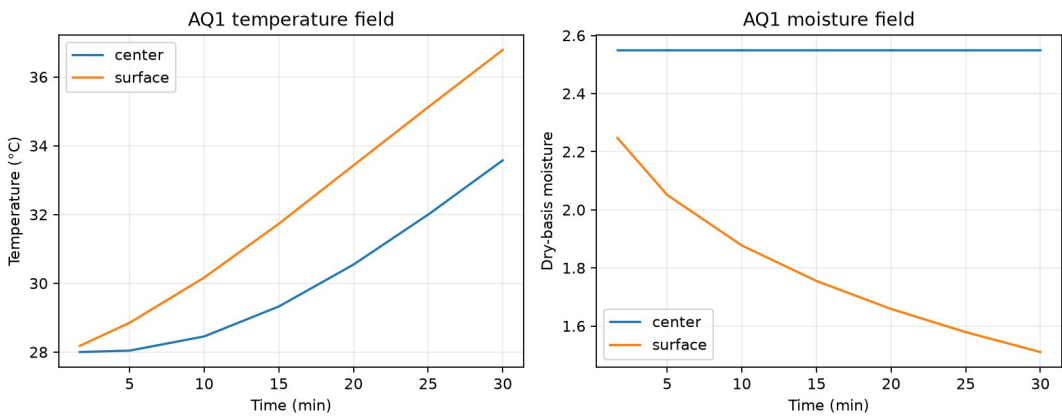


图 5-1 预热阶段温度与含水率的径向分布

从计算结果可以看出：在 1800 s 末，药材中心温度达到 33.5759 °C，表面温度达到 36.7859 °C；表面水分浓度由 2.5500 kg/kg 显著降至 1.5103 kg/kg，而中心水分浓度仍维持在初始 2.55 kg/kg，全域总脱水率达 0.100563。这表明在干燥初期，由于对流传质阻力与内部水分扩散阻力，水分蒸发主要局限于表层区域，形成了显著的内外水分梯度迟滞效应。

### 5.1.5 模型检验或灵敏度分析

在相同 1800 s 终点，正式配置与 2560 单元、0.25 s 配置比较，最大温度差为 0.000477707 K，最大含水率差为 0.000136056 kg/kg。该补充比较支持预热输出对联合加密的稳定性，不能单独据此估计空间或时间收敛阶。总脱水比例 0.100563 为无量纲比例（fraction），按环形面积权重计算；等距半径节点的算术平均会过度计入靠近中心的小面积区域，不能代表整根圆柱体的平均含水率。具体权重及水分收支在第 6 章给出。

## 5.2 问题二：统一经验物性模型与求解

### 5.2.1 建模思路

在实际烘干全过程中，药材的热导率、比热容及水分扩散系数会随着温度升高和水分蒸发发生显著非线性变化 [3, 4]。附录 3 给出了全过程统一经验物性表达式，避免了阶段人工拼接造成的截断误差。本节自  $t=0$  起基于统一物性方程进行全过程连续求解。

### 5.2.2 变量定义与公式推导

问题二保留固定圆柱域、中心对称边界及表面 Robin 边界，温度与含水率控制方程为：

$$\rho(C)c_p(C)\frac{\partial T}{\partial t} = \frac{1}{r}\frac{\partial}{\partial r}(rk(C)\frac{\partial T}{\partial r}),$$

$$\frac{\partial C}{\partial t} = \frac{1}{r}\frac{\partial}{\partial r}(rD(C,T)\frac{\partial C}{\partial r})$$

题面附录 3 给出的物性为 [6]：

$$\rho = 650 + 128C, \quad c_p = 1450 + 2736\frac{C}{C+1}, \quad k = 0.21 + 0.38\frac{C}{C+1},$$

$$D(C,T) = 2.4 \times 10^{-3} \exp\left(-\frac{0.45}{C} - \frac{3850}{T}\right)$$

密度、比热容、导热系数和扩散系数依次使用  $\text{kg/m}^3$ 、 $\text{J}/(\text{kg}\cdot\text{K})$ 、 $\text{W}/(\text{m}\cdot\text{K})$ 、 $\text{m}^2/\text{s}$ 。扩散公式中的温度必须为 K；密度、比热容和导热系数在这组公式中只显式依赖含水率。保留散度形式能够体现不同半径处系数变化产生的通量差，不能将其简化为局部系数乘常系数拉普拉斯算子。

### 5.2.3 求解算法

Step 1: 从均匀初始场重新开始, 使用与问题一一致的空间和时间网格; 不接续问题一的末态。每个时刻按附件 1 插值确定空气状态, 观测窗口外采用末值延拓。

Step 2: 按当前迭代含水率和温度重算物性, 界面采用调和平均, 表面采用半单元与对流串联阻力。沿用问题一的增量、残差双重准则, 循环求解两套三对角系统直至收敛, 不将上一时刻物性冻结整个计算过程。

Step 3: 每秒保留各单元状态, 逐步推进至全域最大含水率首次低于 0.15 kg/kg。提取前 3 h 的半小时节点作为正文表格, 其余逐秒结果继续保存在完整附件中, 保证短期展示与长期终点来自同一套初值问题。

### 5.2.4 结果分析

求解前 3 h (10800 s) 的温湿场演变历程, 表 5-3 和表 5-4 分别列出了每隔 0.5 h 在各径向位置处的温度与水分浓度。

表 5-3 3 小时内药材的温度 (表 3, 单位: °C)

| 时间/h | 0 cm    | 0.5 cm  | 1 cm    | 1.5 cm  | 2 cm    |
|------|---------|---------|---------|---------|---------|
| 0.5  | 32.1900 | 32.3828 | 32.9666 | 33.9614 | 35.4135 |
| 1.0  | 40.3816 | 40.5536 | 41.0605 | 41.8783 | 42.9977 |
| 1.5  | 45.8464 | 45.9345 | 46.1927 | 46.6048 | 47.1398 |
| 2.0  | 48.4500 | 48.4878 | 48.5982 | 48.7733 | 49.0032 |
| 2.5  | 49.4668 | 49.4791 | 49.5135 | 49.5653 | 49.6610 |
| 3.0  | 49.8495 | 49.8553 | 49.8746 | 49.9101 | 49.9664 |

表 5-4 3 小时内药材的水分浓度 (表 4, 单位: kg/kg)

| 时间/h | 0 cm   | 0.5 cm | 1 cm   | 1.5 cm | 2 cm   |
|------|--------|--------|--------|--------|--------|
| 0.5  | 2.5499 | 2.5489 | 2.5255 | 2.3257 | 1.6487 |
| 1.0  | 2.5257 | 2.4948 | 2.3578 | 2.0230 | 1.4711 |
| 1.5  | 2.3860 | 2.3256 | 2.1344 | 1.8020 | 1.3476 |
| 2.0  | 2.1708 | 2.1084 | 1.9236 | 1.6259 | 1.2311 |
| 2.5  | 1.9566 | 1.9006 | 1.7360 | 1.4720 | 1.1167 |
| 3.0  | 1.7662 | 1.7165 | 1.5702 | 1.3333 | 1.0081 |

图 5-2 展示了前 3 小时内药材各径向位置处的温度与含水率剖面变化。

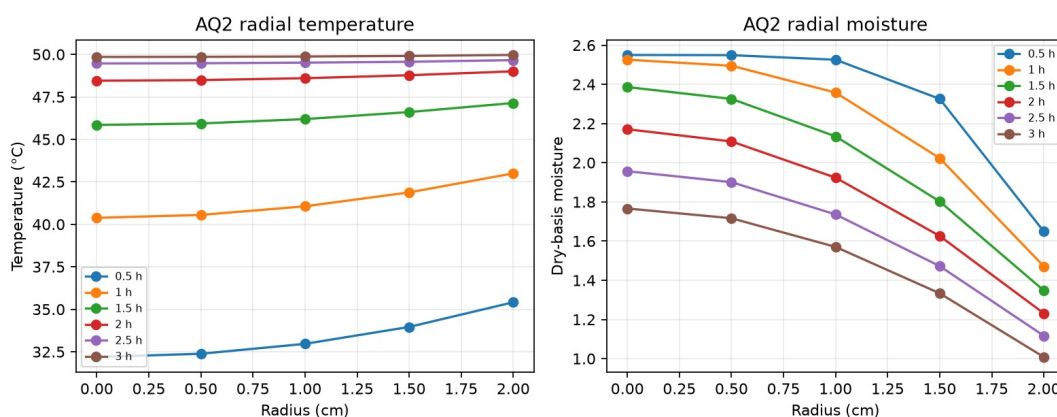


图 5-2 全过程模型前 3 小时的温度与含水率剖面

分析表明：在 3 h 末，药材中心温度升至 49.8495 °C，表面温度升至 49.9664 °C，药材整体温度基本达到烘房气流温度；中心水分浓度降至 1.7662 kg/kg，表面水分浓度降至 1.0081 kg/kg。而在 1800 s 时表面温度为 35.4135 °C，表面水分浓度为 1.6487 kg/kg。这些结果来自同一初边值问题的连续积分，不依赖两段模型的人为拼接。

### 5.2.5 模型检验或灵敏度分析

前 3 h 的温度逐渐接近环境，而含水率仍保持径向梯度，与模型的边界驱动方向一致。问题二与问题一在 1800 s 的表面温度分别为 35.4135 °C、36.7859 °C，含水率分别为 1.6487 kg/kg、1.5103 kg/kg。这是两套物性从初始时刻分别演化的结果，不应强制使两者相等。

全过程积分与问题三使用同一组物性和终点条件，其累计归一水分收支偏差最大为  $4.4757 \times 10^{-14}$ ，实际最多 4 轮迭代，平均约 3.3622 轮。这个结果检验离散水分储存与边界通量的相符程度，并不证明未建模的潜热或解吸机制可以忽略。温度方程残差小仅说明给定经验热容量方程被数值求解；它不等价于整个真实干燥系统的完整能量守恒验证。长程空间、时间和联合加密结果见表 6-1。

## 5.3 问题三：固定半径达标时长模型与求解

### 5.3.1 建模思路

中药材烘干加工的终点判定指标通常要求药材整体水分含量达到安全贮藏标准 [2, 5]。根据工艺要求，以药材全域最大含水率低于 0.1500 kg/kg（计算时遍历所有单元，不预先固定最湿位置）作为烘干完成准则。本节在固定半径（2 cm）条件下连续推进求解，确定给定离散分辨率下的达标节点。

### 5.3.2 变量定义与公式推导

设  $C_* = 0.15 \text{ kg/kg}$  为题面阈值，连续时间的终点定义为：

$$t_{dry} = \inf \{ t > 0 : \max_{0 \leq r \leq R} C(r, t) < C_* \}$$

题面使用“低于”，实际实现保留严格小于关系。离散检测则用所有单元中心值的最大值代替连续空间最大值，并取第一个满足条件的时间节点。当前解的最大含水率在中心附近，但算法不依赖这一位置假设。

### 5.3.3 求解算法

**Step 1:** 使用附录 3 固定域模型从初态连续推进，积分步长为 0.5 s；每一步都计算未舍入单元含水率最大值。输出间隔 60 s 只控制附件保存频率，不改变积分步长。

**Step 2:** 若本步首次满足严格阈值条件，记录本步及前一步时刻和最大含水率，形成宽 0.5 s 的时间夹逼区间。终态独立返回；若不在 60 s 输出节点上，再追加终态。该方法是步进事件检测，未作二分细化，不把线性估计值冒充重新求解的交越时间。达到运行上限但未触发阈值时必须与达标状态区别记录。

**Step 3:** 读取每隔 6 h 的状态并附加终点行，再重构题面要求的物理位置。0.5 s 只表示当前离散轨迹的时间检测分辨率，不是包括空间、经验物性及边界假设在内的总误差界。由于事件采用单元最大值，还需检视中心重构相对第一单元的偏差，不能将离散全域直接等同于连续空间的严格证明。

### 5.3.4 结果分析

逐步检测得到固定几何尺寸下的首次达标节点为 57.1717 h，对应 205818 s。前一步 205817.5 s 的单元最大含水率为 0.150000100797 kg/kg，终点为 0.149999954273 kg/kg，严格跨过 0.15 kg/kg。终点中心重构增量约为  $5.55 \times 10^{-9}$  kg/kg，重构后仍低于阈值；这一核对用于辨别节点与中心的差别，不消除全部空间离散误差。

在烘干结束时刻，中心水分浓度达到临界达标阈值 0.15 kg/kg，真实表面含水率降至 0.0525 kg/kg；该值由边界重构得到，而非外层单元中心值。表 5-5 列出了每隔 6 h 及结束时刻的含水率径向分布。

表 5-5 药材烘干过程的水分浓度（表 5，单位：kg/kg）

| 时间/h                  | 0 cm   | 0.5 cm | 1 cm   | 1.5 cm | 2 cm   |
|-----------------------|--------|--------|--------|--------|--------|
| 6.0                   | 1.0156 | 0.9868 | 0.9006 | 0.7546 | 0.5335 |
| 12.0                  | 0.4548 | 0.4418 | 0.4019 | 0.3289 | 0.1640 |
| 18.0                  | 0.2979 | 0.2906 | 0.2679 | 0.2247 | 0.0842 |
| 24.0                  | 0.2371 | 0.2321 | 0.2161 | 0.1851 | 0.0663 |
| 30.0                  | 0.2053 | 0.2013 | 0.1887 | 0.1637 | 0.0597 |
| 36.0                  | 0.1854 | 0.1821 | 0.1714 | 0.1501 | 0.0565 |
| 42.0                  | 0.1717 | 0.1687 | 0.1593 | 0.1404 | 0.0547 |
| 48.0                  | 0.1615 | 0.1588 | 0.1504 | 0.1332 | 0.0536 |
| 54.0                  | 0.1535 | 0.1511 | 0.1433 | 0.1275 | 0.0528 |
| 烘干结束时间<br>(57.1717 h) | 0.1500 | 0.1477 | 0.1402 | 0.1249 | 0.0525 |

图 5-3 给出了全域最大含水率随时间的演化曲线与 0.15 kg/kg 达标阈值的截断相交过程。

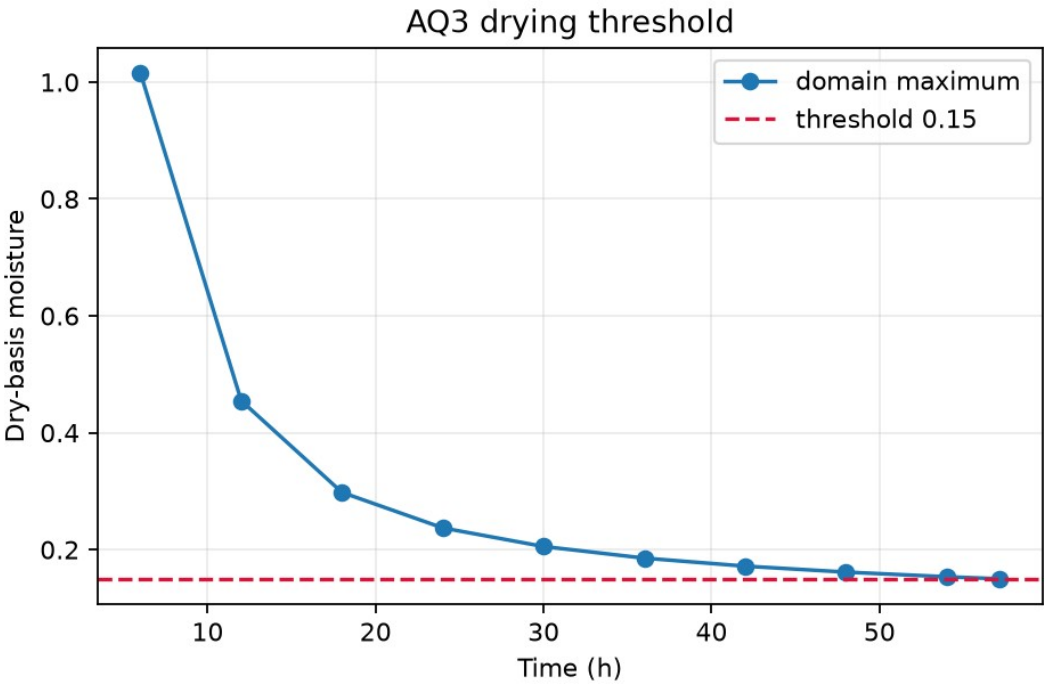


图 5-3 固定半径模型的中心含水率与达标阈值

由图 5-3 可见，早期含水率下降较快，后期逐渐接近阈值。表面先于中心接近低含水率，进一步降低表面水分并不能代替对中心的终点检查。终点表中中心显示为 0.1500 kg/kg，是四位小数舍入的结果，停止检测使用的原始值已严格低于阈值。

### 5.3.5 模型检验或灵敏度分析

最后一小时均值延拓的独立终点为 206934.5 s，比基准增加 0.5425%；等效空气含水率分别乘 0.8 和 1.2 时，终点为 205160.0 s 和 206770.5 s。各运行都保存达标标志及相邻步含水率，没有共用基准结束时刻截断对照。该结果支持所测常值边界变化下时长变化较小；空气含水率并非相对湿度，比例扰动不能被解释为相对湿度改变 20 个百分点。边界情景检验的统一列表与限制见表 6-3。

## 5.4 问题四：收缩域模型与求解

### 5.4.1 建模思路

药材半径随失水变化，固定物理坐标上的计算范围也随之变化。为保留材料层之间的对应关系，假定各层作均匀径向收缩，将位置写成当前半径的比例。题面附录 4 物性与附录 3 不同，本节同时更换物性和几何输入，所得时间差作为综合情景差异进行解释 [6]。

### 5.4.2 变量定义与公式推导

令材料点的无量纲径向坐标为  $\xi = r/R(t)$ ，并定义变换后的场  $\hat{C}(\xi, t) = C(r, t)$ 、 $\hat{T}(\xi, t) = T(r, t)$ ，有：

$$0 \leq \xi \leq 1, \quad v(r, t) = \frac{r \dot{R}(t)}{R(t)} = \xi \dot{R}(t)$$

其中  $v$  为所假定的材料径向速度。链式法则给出固定空间点导数与随材料导数的关系：

$$\left. \frac{\partial C}{\partial t} \right|_r = \left. \frac{\partial \hat{C}}{\partial t} \right|_\xi - \frac{\xi \dot{R}}{R} \frac{\partial \hat{C}}{\partial \xi}, \quad v \frac{\partial C}{\partial r} = \frac{\xi \dot{R}}{R} \frac{\partial \hat{C}}{\partial \xi}$$

两项相加后，材料运动抵消坐标移动项。在等效干基扩散闭合下，物质坐标中的计算方程为：

$$\begin{aligned} \frac{\partial \hat{C}}{\partial t} &= \frac{1}{R(t)^2 \xi} \frac{\partial}{\partial \xi} \left( \xi D \frac{\partial \hat{C}}{\partial \xi} \right), \\ \rho c_p \frac{\partial \hat{T}}{\partial t} &= \frac{1}{R(t)^2 \xi} \frac{\partial}{\partial \xi} \left( \xi k \frac{\partial \hat{T}}{\partial \xi} \right) \end{aligned}$$

这些方程在固定材料坐标上演化，不能在右端额外重复加入网格对流项。若采用静止介质和移动计算网格的另一种假设，则必须重新推导；本文的结果不对应那种物理情形。表面条件也按当前半径换算，例如  $-D \hat{C}_\xi / R = h_m (\hat{C}_s - C_{air})$ 。

附录 4 的经验式为：



$$\rho = 760 + 90C, \quad c_p = 1850 + 2150 \frac{C}{C+1}, \quad k = 0.12 + 0.20 \frac{C}{C+1},$$

$$D(C, T) = 4.2 \times 10^{-4} \exp\left(-\frac{0.30}{C} - \frac{3850}{T}\right)$$

物性单位与问题二相同。这里的经验密度仅用于热容量。为解释干基质量守恒，另引入均匀干骨架密度  $\rho_d$ ：无干物质损失且长度固定时， $\rho_d(t)R(t)^2 = \rho_{d0}R_0^2$ 。由于  $\rho_d$  在当前截面上均匀，其因子可从干基扩散式中约去，得到上述含水率方程。模型因而能够检查归一水分收支，但没有独立求解孔隙率、应力或变形本构，不据此宣称完整多相耦合已经获得验证。

### 5.4.3 求解算法

Step 1: 在  $0 \leq \xi \leq 1$  上划分 1280 个单元，以 0.5 s 步长推进。在每一步的新时刻按附件 2 线性插值得到半径，既不使用样条，也不需要数值计算半径导数。半径的轻微实测起伏被保留，不能以数值方便为理由单调化原始数据。

Step 2: 以当前半径将内部扩散、导热系数分别缩放为  $D/R^2$ 、 $k/R^2$ ，重新计算表面半单元阻力。归一化含水率边界系数为：

$$\gamma_{C,\xi} = \frac{1}{R} \left( \frac{1}{h_m} + \frac{RA\xi}{2D_N} \right)^{-1}$$

Step 3: 每步按与固定域相同的增量和残差准则完成物性迭代，再检查单元最大含水率。保存首次达标节点及前一节点，按 60 s 间隔和独立终态输出分布。内部计算用未舍入值，电子表格才格式化为四位小数。

Step 4: 后处理时把单元中心位置乘以当前半径，还原到物理坐标。对落在当前药材外部的的位置保留空值，并另外给出真实表面列。否则把域外位置误作零值，会造成“外层提前完全脱水”的假象。

### 5.4.4 结果分析

数值求解结果表明，在实测收缩与附录 4 物性共同作用的情景下，首次达标时间为 50.8249 h，对应 182969.5 s，相较于固定半径模型的 57.1717 h 耗时缩短了 11.1013 %。

在烘干达标时刻，药材半径为 1.2 cm，中心含水率显示为 0.1500 kg/kg，真实表面为 0.0525 kg/kg。内部单元最小含水率约 0.0535 kg/kg，位置在最外层单元中心，与表面列不是同一采样点。前一步 182969.0 s 的单元最大值为 0.150000001603 kg/kg，本步为 0.149999743274 kg/kg。表 5-6 列出每隔 6 h 及终点在固定物理位置和真实表面上的分布，域外位置不填零。综合时长缩短约 11.1%，精确到四位小数为 11.1013%。

表 5-6 药材烘干过程的水分浓度（表 6，单位：kg/kg）

| 时间/h               | 0 cm   | 0.5 cm | 1 cm   | 1.5 cm | 2 cm | 药材表面   | 当前半径/<br>cm |
|--------------------|--------|--------|--------|--------|------|--------|-------------|
| 6.0                | 1.7172 | 1.5357 | 1.0217 |        |      | 0.4217 | 1.3740      |
| 12.0               | 0.7344 | 0.6518 | 0.4069 |        |      | 0.1667 | 1.2480      |
| 18.0               | 0.4066 | 0.3668 | 0.2388 |        |      | 0.0889 | 1.2140      |
| 24.0               | 0.2837 | 0.2599 | 0.1783 |        |      | 0.0671 | 1.2040      |
| 30.0               | 0.2253 | 0.2085 | 0.1488 |        |      | 0.0593 | 1.2010      |
| 36.0               | 0.1921 | 0.1790 | 0.1312 |        |      | 0.0558 | 1.2000      |
| 42.0               | 0.1706 | 0.1598 | 0.1196 |        |      | 0.0539 | 1.2000      |
| 48.0               | 0.1556 | 0.1463 | 0.1113 |        |      | 0.0529 | 1.2000      |
| 烘干结束时间 (50.8249 h) | 0.1500 | 0.1413 | 0.1081 |        |      | 0.0525 | 1.2000      |

图 5-4 展示了收缩情景下药材半径和内部含水率的演化历程。

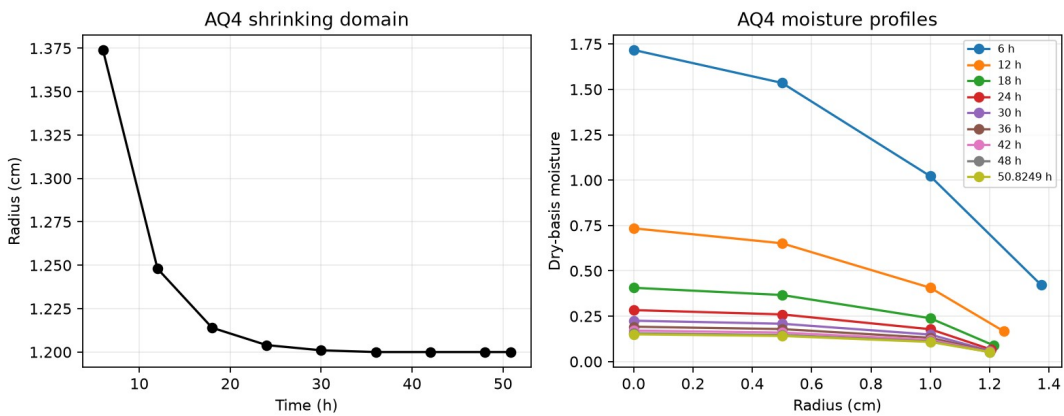


图 5-4 收缩情景的半径与径向含水率演化

表 5-6 的空白格表示相应位置已在药材外部；真实表面与固定 1 cm 位置不能互相替代。图 5-4 呈现收缩半径与不同位置的含水率演化，应结合当时的几何域理解曲线。问题四终点比问题三早，但这尚不足以得出“全部缩短由几何收缩造成”的结论。

5.4.5 模型检验或灵敏度分析

当前固定域和移动域共用同一通量求解内核，恒半径是输入半径序列取常数的特例。因此本文不沿用旧版两套求解器的退化误差作为新代码的独立验证。直接对动态收缩完整轨迹作联合加密，最大温度差 0.000474706 K、最大含水率差 0.000322393 kg/kg、时长相对差 0.000820%，均

达到预设目标。另以附录 4 物性在固定半径下独立求至 464739.5 s，得到 129.0943 h；三格顺序对照见表 6-2，该算例不能替代缺失的第四组合。

## 6 模型检验与灵敏度分析

### 6.1 长程网格与时间步验证

数值可靠性需要覆盖接近终点的低含水率阶段，而不能只检查预热末态。本文以 0.05 K 的温度场差、0.0005 kg/kg 的含水率场差及 0.5% 的时长相对差为预设比较目标。细网格场插值到粗网格单元中心，在两条轨迹共同的 60 s 输出时刻计算最大绝对差；时长差以较细配置终点为分母。该定义覆盖保存的共同轨迹，不声称检查了每个内部时间步的连续误差。

最初 80 单元配置未通过含水率场目标。在 0.5 s 下逐次增加空间单元，问题三的 160 对 320、320 对 640、640 对 1280 单元比较，含水率最大差分别为 0.002679、0.001643 和 0.000917 kg/kg，均超过目标。直到 1280 对 2560 单元时降至 0.000452227 kg/kg，才满足要求。即使粗网格的干燥时间接近，也不能用终点指标掩盖外层剖面偏差。加密过程保留失败记录，未放宽接受标准。

表 6-1 为长程网格与时间步验证。空间比较保持 0.5 s；时间比较在 2560 单元下将 0.5 s 减至 0.25 s；联合比较将正式 1280 单元、0.5 s 配置改为 2560 单元、0.25 s。

表 6-1 长程网格与时间步验证

| 模型  | 比较类别 | 最大温度差/K     | 最大含水率差/(kg/kg) | 时长相对差/%  |
|-----|------|-------------|----------------|----------|
| 问题三 | 空间   | 0.000000501 | 0.000452227    | 0.002186 |
| 问题三 | 时间   | 0.000435363 | 0.000123114    | 0.000486 |
| 问题三 | 联合   | 0.000435354 | 0.000452145    | 0.002672 |
| 问题四 | 空间   | 0.000000772 | 0.000057145    | 0.000547 |
| 问题四 | 时间   | 0.000474801 | 0.000276280    | 0.000273 |
| 问题四 | 联合   | 0.000474706 | 0.000322393    | 0.000820 |

表 6-1 中两问联合加密均满足预设目标。温度的空间差远小于时间差，而问题三含水率差主要受空间分辨率约束。后向欧拉仍为一阶时间格式；比较是实际离散解之间的一致性证据，不是解析真解误差或实验误差。主终点夹逼宽 0.5 s 仅描述当前轨迹的事件定位，不能替代离散配置改变产生的时长差。

## 6.2 迭代与水分收支

非线性步同时满足增量与最终方程残差准则才被接受。问题三、问题四运行最多均 4 轮，平均分别约 3.3622 轮、3.4820 轮；最大含水率方程归一残差分别为  $1.0867 \times 10^{-11}$  和  $1.4650 \times 10^{-11}$ 。50 轮是失败保护上限，不是每步固定更新次数。

令  $w_i = \xi_{i+1/2}^2 - \xi_{i-1/2}^2$ ，权重之和为 1。干基平均含水率及连续收支关系为：

$$\bar{C}^n = \sum_{i=1}^N w_i C_i^n, \quad \frac{d\bar{C}}{dt} = -\frac{2h_m}{R(t)}(C_s - C_{air})$$

固定长度、无干物质流失时，干骨架密度随半径平方的倒数增加，材料坐标权重保持不变。这里以固定干物质质量统计含水量，并非把湿体积缩小直接当作水分流失。记每步实际水分矩阵使用的表面系数为  $\gamma_C^n$ ，离散累计检查量为：

$$E_w^n = \frac{|\bar{C}^n - C_0 + \sum_{j=1}^n \frac{2\gamma_C^j}{R^j}(C_N^j - C_{air}^j)\Delta t_j|}{C_0}$$

此式采用实际求解该步的矩阵系数，不以事后重组的系数替代，避免把重组差异误计为通量。问题三、问题四最大累计归一偏差分别为  $4.4757 \times 10^{-14}$ 、 $4.7718 \times 10^{-14}$ ，全部长程验证运行均小于  $6 \times 10^{-14}$ 。这说明离散水分储存与外排通量相符，不能验证经验扩散系数或未建模潜热的真实性。

实现层面还检查非整秒终点采样、均匀场收缩、常半径退化、输出节点间事件、迭代失败及未达标状态。常半径测试检查共用内核两种接口的一致性，独立性较弱；均匀场测试检查无驱动力时是否产生虚假源项。不同测试各针对一种失效方式，不能将通过标志解释为实验预测精度评分。

## 6.3 三格顺序对照与作用路径

保持初值、外界环境、网格及阈值一致，增加附录 4 物性配固定半径的中间运行。表 6-2 为三格顺序对照，每个终点均由各自完整轨迹独立检测。

表 6-2 三格顺序对照

| 情景   | 经验物性 | 半径条件    | 达标时间/s   | 达标时间/h   |
|------|------|---------|----------|----------|
| 问题三  | 附录 3 | 固定 2 cm | 205818.0 | 57.1717  |
| 中间对照 | 附录 4 | 固定 2 cm | 464739.5 | 129.0943 |
| 问题四  | 附录 4 | 附件 2 收缩 | 182969.5 | 50.8249  |

表 6-2 表明先更换物性延长时间，随后引入给定收缩曲线缩短时间。以问题三时间为统一分母，两步变化分别为+125.8012%、-136.9025%，相加为-11.1013%。第二项小于-100%不表示从中间时间出发缩短超过 100%，因为分母仍是问题三时间。该口径使两项可加，解释必须保留分母及改变顺序。

两套扩散关系在相同温度与含水率下满足：

$$\frac{D_4}{D_3} = 0.175 \exp\left(\frac{0.15}{C}\right)$$

在  $C \geq 0.15 \text{ kg/kg}$  范围内比值不超过约 0.476。收缩至 1.2 cm 则使  $1/R^2$  相较 2 cm 增加约 2.78 倍。这是同状态的公式比较，不能直接换算完整时长贡献；表层含水率可低于 0.15 kg/kg，因而该比值上界不能无条件推广至全部单元。温湿场演化、边界阻力和热物性差异共同决定实际终点。

三格计算缺少“附录 3 物性、收缩半径”组合，因此不能估计交互项，也不能将两步时长差解释为与路径无关的因果贡献。当前证据支持指定模型路径的情景比较，尚不支持向其他材料迁移的独立物理归因。

### 6.4 边界情景检验与不确定性

基准在 14400 s 后保持末值。低、高情景将整条空气含水率序列乘 0.8、1.2，温度不变；最后一小时均值情景保留观测段，随后改用 10800—14400 s 的时间平均。各情景运行至独立终点，不以基准结束时刻截断。表 6-3 给出边界情景检验。

表 6-3 边界情景检验

| 环境设定         | 达标节点/s   | 相对基准变化/% |
|--------------|----------|----------|
| 末值延拓基准       | 205818.0 | 0.0000   |
| 等效空气含水率乘 0.8 | 205160.0 | -0.3197  |
| 等效空气含水率乘 1.2 | 206770.5 | +0.4628  |
| 最后一小时时间均值延拓  | 206934.5 | +0.5425  |

表 6-3 所测时长变化均小于 0.55%，但高、低情景不对称，反映非线性扩散与边界驱动共同作用。空气等效含水率不是相对湿度，不能由这些确定性情景推断所有环境都不敏感，也不能将其视为概率置信区间。

最后一小时情景在单元达标节点的中心二次重构约为 0.150000001002 kg/kg，略高于阈值。约  $10^{-9} \text{ kg/kg}$  的差异说明表中时间严格说是单元判据节点，并不是连续中心严格达标的证明。本文保留这一差别，不修改结果以消除表面不一致；要求更严格连续空间判据时，应进一步细化事件和空间重构。相较于 0.5425% 的情景时长变化，差异很小，但概念仍需区分。

剩余不确定性主要来自经验物性、等效边界闭合、未建模潜热与一维均匀收缩假设。没有独立内部温湿场观测，无法将模型偏差分解成可识别概率分布。数值差和情景差不能改称实测误差或工艺安全裕量。

## 7 模型评价与推广

### 7.1 模型优点与结果的适用范围

本文把题目的输出位置、采样间隔和全域终点条件放在同一个径向数值框架内处理。圆柱壳层的面积、体积直接进入有限体积式，界面系数采用调和平均，边界串联半单元与对流阻力，使几何、内部传输和外部交换关系具有清晰的对应。三对角求解避免构造稠密矩阵，适于长时间细步长积分 [1]。

四问之间的计算关系也得到明确区分：问题一与问题二各自从初始场出发，问题二的短期表格和问题三的长期终点使用同一经验物性，问题四则明确更换几何和物性。这样的组织方式能够避免把不同模型的末态拼接起来，并帮助判断某个结果差异来自哪一类设定。

预热阶段结果显示中心尚湿而表层已明显失水，全过程结果进一步显示温度先趋于平衡、含水率后趋于达标。因此工程判断应关注全域最大含水率，而不只看表面状态或环境温度。两种主情景的 57.1717 h 与 50.8249 h 是已声明边界和物性下的数值预测，可用于比较给定情景，不能直接作为所有药材品种的工艺常数。

### 7.2 模型局限与改进顺序

首先，宏观扩散近似没有分离多相迁移，也未显式求解蒸发潜热和干燥应力。空气与药材浓度的等效边界闭合缺少吸附平衡关系支持，实测标定后才适合扩大应用范围 [3, 5]。一维径向假设对中段较合理，但对两端区域和短粗物体需要增加轴向分辨率。

其次，物质坐标模型采用均匀径向收缩，干骨架密度变化由干物质守恒约束给出，但未求解变形本构与孔隙结构。三格顺序比较补充了物性改变后的固定域终点，却仍缺少第四组合，不能识别交互。输入半径曲线与计算几何一致是模型设定，不是独立预测验证；需要不同批次的内部含水率观测检验收缩假设。

最后，当前计算已记录非线性增量、残差、事件夹逼和水分收支，并完成长期联合加密。剩余数值限制包括用单元最大值近似连续全域最大值、共同 60 s 时刻的场比较，以及后向欧拉的一阶时间误差。提高精度应同时调整事件重构与空间分辨率；补充实测参数与内部观测则可检验模型结构偏差，不能以增加网格数量替代物理验证。未执行实验和缺少的材料参数不作为已有成果。

### 7.3 应用与推广建议

该框架可作为给定环境和几何条件下的离线模拟工具，用于查看内部梯度和比较候选运行情景。若要用于在线停止控制，需要额外的传感器校准、模型偏差评估与工艺安全裕量；现有数据不能直接计算药效保留率、能耗下降幅度或经济收益。对其他柱状食品或农产品，也应先重新确定物性、尺寸和边界关系，不能只替换名称便沿用参数 [4, 5]。

### AI 工具使用声明

本参赛队在竞赛过程中使用了 AI 工具，主要用于代码调试、结果核对、模型与数值验证讨论、论文修订及排版辅助，详细使用情况见支撑材料。

## 8 参考文献

- [1] Patankar S V. Numerical Heat Transfer and Fluid Flow [M]. Hemisphere Publishing Corporation, 1980.
- [2] Crank J. The Mathematics of Diffusion [M]. 2nd ed. Oxford: Clarendon Press, 1975.
- [3] Whitaker S. Simultaneous heat, mass, and momentum transfer in porous media: A theory of drying [J]. Advances in Heat Transfer, 1977, 13: 119-203. DOI: 10.1016/S0065-2717(08)70223-5.
- [4] Mujumdar A S, ed. Handbook of Industrial Drying [M]. 4th ed. CRC Press, 2014. DOI: 10.1201/b17208.
- [5] Defraeye T. Advanced computational modelling for drying processes: A review [J]. Applied Energy, 2014, 131: 323-344. DOI: 10.1016/j.apenergy.2014.06.027.
- [6] 2026 年高教社杯全国大学生数学建模竞赛. A 题：药材的烘干问题及附件 [Z]. 本项目提供的赛题材料, 2026.

## 附录

### 附录 A：数据与求解复现说明

正文计算来自 `paper_output/code/modeling/` 下的 `q1_model.py` 至 `q4_model.py`，共同调用 `core_solver.py`。在仓库根目录、满足 `requirements.txt` 依赖的 Python 环境中运行 `python paper_output/code/modeling/run_modeling.py` 可生成模型输出。运行清单 `paper_output/results/run_manifest.json` 记录脚本、输入、输出与退出码，复算时应核对其中的文件摘要。

初始场为 301.15 K 和 2.55 kg/kg，正式空间网格 1280 单元、时间步长 0.5 s，最多 50 轮迭代并按增量及残差停止。求解器函数默认参数仅供通用调用，正式四问从验证报告读取所选配置，复现不能省略这一层配置读取。环境和半径分别读取 `paper_output/data_cleaned/a_environment.csv`、`a_radius.csv`，采用分段线性插值。三对角算法为 `core_solver.py` 中的 `solve_thomas`。

`result1.xlsx` 含温度与水分浓度两表，从 1 至 1800 s 逐秒记录；`result2.xlsx` 同样有两表，从 1 s 延续至 205818 s，表 3、表 4 仅是前 3 h 的展示切片。`result3.xlsx` 按 60 s 保存固定半径含水率并额外保存 205818 s 终点；`result4.xlsx` 按 60 s 保存收缩域含水率并额外保存 182969.5 s 终点，最后一列为真实表面。四个文件均位于 `paper_output/tables/`。域外空值表示几何上不适用，不作为数值失败，也不填补为零。

### 附录 B：正文表格与证据文件对应

表 4-2、表 4-3 分别整理自 `table_data_profile_a_environment`、`table_data_profile_a_radius` 对应的 CSV。本文将重复单位、类型字段和开尔文转换列合并为中文表头，保留样本数、缺失数和统计量含义。

表 5-1 至表 5-6 依次对应 `table1` 至 `table6`。文件为 `table_aq1_temperature.csv`、`table_aq1_moisture.csv`、`table_aq2_temperature.csv`、`table_aq2_moisture.csv`、`table_aq3_drying_moisture.csv`、`table_aq4_shrinkage_moisture.csv`，均位于 `paper_output/tables/`，正文逐行与最新 CSV 同步。表 6-1、表 6-2、表 6-3 分别对应 `table7`、`table8`、`table9`，整理自完整数值验证记录，正文仅作有效位舍入。六幅图见 `paper_output/figures/`。

### 附录 C：数值检验的追溯

数值对照来自 `paper_output/qa/model_validation_checks.json`，生成程序为 `paper_output/code/visualization/run_numerical_validation.py`。记录包括 80 单元起始对照、逐级加密、1280 单元正式设置与 2560 单元参考设置、0.5 与 0.25 s 时间比较、三格顺序对照及三个独立



边界情景。问题一补充联合加密见 `paper_output/qa/delivery_audit.json` 的预热比较记录。数据检验和后处理不产生新实测样本。

支撑材料应包含以下文件组：模型与结果导出源码、数据整理源码及通用读取模块、可视化和验证源码、运行依赖说明、两份清洗 CSV、四份结果 Excel、六张主结果 CSV 及数据统计表、六幅图、模型结果与指标结论 JSON、数值验证 JSON、运行清单，以及 AI 工具使用详情.pdf。完整源程序列于附录 E，文件名与相对目录均保留以支持复算。原始赛题数据由主办方提供，不作为自主查阅数据重复解释。

AI 使用详情应如实记录工具名称、用途、提示过程和采纳方式。本次修订由 Codex 辅助核对文件、修订论述和排版；自动检查不等价于参赛队的逐项人工审查。工具具体模型版本及此前使用历史应由参赛队据真实记录补充，不以未知版本冒充已确认信息。

附录 D：完整运行诊断

以下为各问实际运行的诊断摘要，数值保留原始精度或六位有效数字。状态 0 表示到达指定展示终点，状态 1 表示触发干燥阈值；负状态表示失败，不能作为结果。精确原始值以运行 JSON 为准，正文保留解释所需的有效位。

| 诊断量                     | 数值                     | 单位    |
|-------------------------|------------------------|-------|
| aq1_left_time_s         | 1799.5                 | s     |
| aq1_end_time_s          | 1800.0                 | s     |
| aq1_left_max_C          | 2.5499922976882003     | kg/kg |
| aq1_final_max_C         | 2.549992271962013      | kg/kg |
| aq1_water_balance       | 1.2060069718477192e-14 | 1     |
| aq1_step_water_balance  | 2.00152224587275e-16   | 1     |
| aq1_residual_C          | 3.8706802378437625e-16 | 1     |
| aq1_residual_T          | 2.4453460187734737e-16 | 1     |
| aq1_max_iterations      | 4.0                    | count |
| aq1_mean_iterations     | 4.0                    | count |
| aq1_steps               | 3600                   | count |
| aq1_center_excess_bound | 1.59003e-10            | kg/kg |
| aq1_status_code         | 0.0                    | 1     |
| aq2_left_time_s         | 205817.5               | s     |
| aq2_end_time_s          | 205818.0               | s     |
| aq2_left_max_C          | 0.15000010079669182    | kg/kg |
| aq2_final_max_C         | 0.14999995427300694    | kg/kg |
| aq2_water_balance       | 4.4757226247633767e-14 | 1     |
| aq2_step_water_balance  | 2.931943095510862e-16  | 1     |
| aq2_residual_C          | 1.0867260945845985e-11 | 1     |

| 诊断量                     | 数值                     | 单位    |
|-------------------------|------------------------|-------|
| aq2_residual_T          | 3.512785547753425e-16  | 1     |
| aq2_max_iterations      | 4.0                    | count |
| aq2_mean_iterations     | 3.36223994014129       | count |
| aq2_steps               | 411636                 | count |
| aq2_center_excess_bound | 1.54901110160921e-07   | kg/kg |
| aq2_status_code         | 1.0                    | 1     |
| aq3_left_time_s         | 205817.5               | s     |
| aq3_end_time_s          | 205818.0               | s     |
| aq3_left_max_C          | 0.15000010079669182    | kg/kg |
| aq3_final_max_C         | 0.14999995427300694    | kg/kg |
| aq3_water_balance       | 4.4757226247633767e-14 | 1     |
| aq3_step_water_balance  | 2.931943095510862e-16  | 1     |
| aq3_residual_C          | 1.0867260945845985e-11 | 1     |
| aq3_residual_T          | 3.512785547753425e-16  | 1     |
| aq3_max_iterations      | 4.0                    | count |
| aq3_mean_iterations     | 3.36223994014129       | count |
| aq3_steps               | 411636                 | count |
| aq3_center_excess_bound | 1.54901110160921e-07   | kg/kg |
| aq3_status_code         | 1.0                    | 1     |
| aq4_left_time_s         | 182969                 | s     |
| aq4_end_time_s          | 182969.5               | s     |
| aq4_left_max_C          | 0.15000000160267352    | kg/kg |
| aq4_final_max_C         | 0.14999974327394916    | kg/kg |
| aq4_water_balance       | 4.7717820979967513e-14 | 1     |
| aq4_step_water_balance  | 1.285284472465843e-16  | 1     |
| aq4_residual_C          | 1.4649702830163243e-11 | 1     |
| aq4_residual_T          | 3.347897338265732e-16  | 1     |
| aq4_max_iterations      | 4.0                    | count |
| aq4_mean_iterations     | 3.482017494719065      | count |
| aq4_steps               | 365939                 | count |
| aq4_center_excess_bound | 2.4056242708958564e-07 | kg/kg |
| aq4_status_code         | 1.0                    | 1     |

## 附录 E：完整可运行源程序

以下逐文件列出本次建模、结果导出、图形生成与数值验证的源程序。保持相对目录关系，在项目根目录执行附录 A 的复现入口。数据整理程序还调用随项目提供的通用表格读取模块，该模块一并列出。

### E.1 paper\_output/code/data\_processing/prepare\_a\_data.py

```
"""S3 only: inspect A inputs, convert units, plot observations, emit handoffs."""
import hashlib
import json
from datetime import datetime, timezone
from pathlib import Path
import sys
import time

ROOT = Path(__file__).resolve().parents[3]
OUT = ROOT / 'paper_output'
sys.path.insert(0, str(ROOT / '.agents/skills/data-cleaning-and-visualization/scripts'))
import robust_loader as loader
import numpy as np
from openpyxl import load_workbook
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt

def digest(path):
    return hashlib.sha256(path.read_bytes()).hexdigest()

def save_json(name, data):
    path = OUT / name
    path.parent.mkdir(parents=True, exist_ok=True)
    path.write_text(json.dumps(data, ensure_ascii=False, indent=2, allow_nan=False) +
'\n', encoding='utf-8', newline='\n')

def main():
    started = time.perf_counter()
    stamp = datetime.now(timezone.utc).isoformat()
    manifest_path = OUT / 'input_manifest.json'
    manifest = json.loads(manifest_path.read_text(encoding='utf-8'))
    analysis = json.loads((OUT /
'step1/problem_analysis.json').read_text(encoding='utf-8'))
    route = json.loads((OUT / 'plan/model_route.json').read_text(encoding='utf-8'))
    allowed = {d['path'] for d in analysis['data_files'] if d['role'] == 'raw_data'}
    entries = [e for e in manifest['entries'] if e['path'] in allowed]
    assert len(entries) == 2
    assert all(e['role'] == 'raw_data' and e['usable_for_modeling'] for e in entries)
    assert all(digest(ROOT / e['path']) == e['sha256'] for e in entries)
    diagnostics = [loader.inspect_xlsx(ROOT / e['path']) for e in entries]
    report = dict(schema_version='1.0',
generated_by='paper_output/code/data_processing/prepare_a_data.py',
generated_at=stamp, status='PASS' if all(d['readable'] for d in
diagnostics) else 'FAIL',
input_manifest_used=True, input_manifest_sha256=digest(manifest_path),
data_files=diagnostics, selected_problem='A',
skipped_files=[{'path': e['path'], 'role': e['role'], 'reason':
'outside_A_raw_data'}
for e in manifest['entries'] if e['path'] not in
allowed],
errors=[err for d in diagnostics for err in d['errors']])
save_json('data_cleaned/load_report.json', report)
assert report['status'] == 'PASS'
```

```

arrays, datasets = [], []
settings = [
    ('a_environment.csv', ['时间', '温度', '水分浓度'],
     ['time_s', 'temperature_C', 'air_moisture_kgkg', 'temperature_K'], ['s', 'degC',
     'kg/kg', 'K'], 60, 14400),
    ('a_radius.csv', ['时间', '半径'], ['time_s', 'radius_cm', 'radius_m'], ['s', 'cm',
     'm'], 1800, 259200),
]
for entry, (name, headers, columns, units, step, end) in zip(entries, settings):
    wb = load_workbook(ROOT / entry['path'], read_only=True, data_only=True)
    rows = list(wb['Sheet1'].values)
    wb.close()
    assert list(rows[0]) == headers
    values = np.asarray(rows[1:], dtype=float)
    assert np.isfinite(values).all(), 'Missing or non-finite observation: do not
impute automatically'
    assert values[0, 0] == 0 and values[-1, 0] == end
    assert np.all(np.diff(values[:, 0]) == step)
    assert (values[:, 1:] > 0).all()
    converted = np.column_stack([values, values[:, 1] + 273.15 if end == 14400 else
values[:, 1] / 100])
    path = OUT / 'data_cleaned' / name
    with path.open('w', encoding='utf-8', newline='\n') as stream:
        np.savetxt(stream, converted, delimiter=',', header=''.join(columns),
comments='', fmt='%.12g')
    check = np.loadtxt(path, delimiter=',', skiprows=1)
    np.testing.assert_allclose(check, converted, rtol=0, atol=1e-10)
    arrays.append(converted)
    datasets.append(dict(path=entry['path'], sha256=entry['sha256'], role='raw_data',
cleaned_output=path.relative_to(ROOT).as_posix(),
cleaned_sha256=digest(path),
rows=len(values), columns=columns, units=dict(zip(columns,
units)),
missing_values=0, duplicate_times=0, removed_rows=0,
imputed_values=0,
time_start_s=0, time_end_s=end, interval_s=step,
min_values=dict(zip(columns,
converted.min(axis=0).tolist())),
max_values=dict(zip(columns,
converted.max(axis=0).tolist())),
question_ids=['AQ1', 'AQ2', 'AQ3', 'AQ4'] if end == 14400
else ['AQ4'])))

env, rad = arrays
tail = env[env[:, 0] >= 10800]
environment = dict(interpolation='piecewise_linear', support_s=[0, 14400],
extension='hold_last', last_temperature_C=float(env[-1, 1]),
last_Ce=float(env[-1, 2]), sensitivity_window_s=[10800, 14400],
tail_mean_temperature_C=float(np.trapezoid(tail[:, 1], tail[:,
0]) / 3600), tail_mean_C=float(np.trapezoid(tail[:, 2], tail[:, 0]) / 3600),
tail_std_temperature_C=float(tail[:, 1].std(ddof=1)),
tail_std_Ce=float(tail[:, 2].std(ddof=1)),
extension_is_observed=False)
radius = dict(interpolation='piecewise_linear', support_s=[0, 259200],
extension='hold_last',
last_radius_m=float(rad[-1, 2]),
nonincreasing=bool(np.all(np.diff(rad[:, 2]) <= 0)),
increasing_intervals=int(np.count_nonzero(np.diff(rad[:, 2]) > 0)),
extension_is_observed=False)
report.update(summary=dict(data_file_count=2, readable_data_file_count=2,
total_observations=len(env)+len(rad)),
quality=datasets, boundary_statistics=environment,
radius_statistics=radius)
save_json('data_cleaned/load_report.json', report)

figdir = OUT / 'figures'
figdir.mkdir(exist_ok=True)
plt.rcParams.update({'font.size': 10, 'axes.spines.top': False, 'axes.spines.right':
False})

```

```

fig, axes = plt.subplots(2, 1, figsize=(8, 6), sharex=True, layout='constrained')
axes[0].plot(env[:, 0]/3600, env[:, 1], color='#16697a', lw=1.5)
axes[0].set(ylabel='Air temperature (deg C)', title='Observed chamber inputs: 0-4 h')
axes[1].plot(env[:, 0]/3600, env[:, 2], color='#c06c35', lw=1.5)
axes[1].set(xlabel='Time (h)', ylabel='Air moisture (kg/kg)')
for ax in axes:
    ax.grid(alpha=0.2)
fig.savefig(figdir / 'fig_a_environment.png', dpi=180)
plt.close(fig)
fig, ax = plt.subplots(figsize=(8, 4), layout='constrained')
ax.plot(rad[:, 0]/3600, rad[:, 1], color='#16697a', lw=1.5)
ax.scatter(rad[:, 6, 0]/3600, rad[:, 6, 1], s=12, color='#16697a')
ax.set(xlabel='Time (h)', ylabel='Radius (cm)', title='Observed radius: 0-72 h')
ax.grid(alpha=0.2)
fig.savefig(figdir / 'fig_a_radius.png', dpi=180)
plt.close(fig)

figure_items = []
for fid, title, source, qids in [
    ('fig_a_environment', '烘房温度与水分浓度实测输入', datasets[0],
    ['AQ1', 'AQ2', 'AQ3', 'AQ4']),
    ('fig_a_radius', '药材半径实测输入', datasets[1], ['AQ4'])]:
    path = OUT / 'figures' / (fid + '.png')
    figure_items.append(dict(figure_id=fid, title=title,
    path=path.relative_to(ROOT).as_posix(),
    expected_path=path.relative_to(ROOT).as_posix(),
    question_ids=qids,
    source_data=source['cleaned_output'],
    source_sha256=source['cleaned_sha256'],
    sha256=digest(path), bytes=path.stat().st_size,
    status='generated',
    ok=True, exists=True, placeholder=False,
    evidence_type='input_observation',
    interpretation='展示官方输入·不是模型预测或独立验证。',
    paper_section='2 数据与问题分析'))
    future = {f['figure_id']: f for q in route['questions'] for f in q['figures']}
    save_json('figure_index.json', dict(schema_version='1.0', generated_at=stamp,
    figures=figure_items))
    save_json('plan/visualization_plan.json', dict(schema_version='1.0',
    generated_at=stamp,
    figures=figure_items, future_model_figures=list(future.values()),
    note='只有实际生成的S3输入图进入figure_index;模型图待S5。'))
    save_json('plan/data_plan.json', dict(schema_version='1.0', generated_at=stamp,
    selected_problem='A',
    input_manifest_sha256=digest(manifest_path),
    model_route_sha256=digest(OUT/'plan/model_route.json'),
    data_files=datasets, environment=environment, radius=radius,
    cleaning_policy='检查全表;不填补、不平滑、不去趋势,保留原始观测波动;只增加SI单
    位列。',
    interpolation_policy='S4 按本计划在计算时插值, S3 清洗文件只保留实测节点。',
    output_sampling={'AQ1': '1 s, 0.1 cm, through 1800 s',
    'AQ2': '1 s, 0.1 cm, through AQ3 event; tables at 0.5 h
    through 3 h',
    'AQ3': '60 s, 0.1 cm, append event',
    'AQ4': '60 s, valid physical 0.1 cm nodes and true surface;
    append event'},
    external_validation={'dataset_acquired': False,
    'required_for_numerical_execution': False},
    figure_ids=[f['figure_id'] for f in figure_items]))

    outputs = [OUT/'data_cleaned/load_report.json', OUT/'plan/data_plan.json',
    OUT/'plan/visualization_plan.json', OUT/'figure_index.json']
    outputs += [OUT/'data_cleaned'/s[0] for s in settings]
    outputs += [ROOT/f['path'] for f in figure_items]
    tracked_inputs = [manifest_path, OUT/'step1/problem_analysis.json',
    OUT/'plan/model_route.json']
    tracked_inputs += [ROOT/e['path'] for e in entries]
    record = lambda p: dict(path=p.relative_to(ROOT).as_posix(), sha256=digest(p),
    bytes=p.stat().st_size)
    save_json('data_cleaned/data_pipeline_run.json', dict(schema_version='1.0',

```

```

stage='S3', status='PASS',
        generated_at=stamp, exit_code=0, elapsed_s=time.perf_counter()-started,
        command=f'{sys.executable} -B
paper_output/code/data_processing/prepare_a_data.py',
        executable=sys.executable, python=sys.version, numpy=np.__version__,
matplotlib=matplotlib.__version__,
        scripts=[record(Path(__file__)), record(Path(loader.__file__))],
        inputs=[record(p) for p in tracked_inputs], outputs=[record(p) for p in
outputs],
        note='数据清理运行据；不是 S5 模型运行记录。')
print(json.dumps({'stage': 'S3', 'status': 'PASS', 'rows': [len(env),len(rad)],
        'figures': len(figure_items), 'environment': environment, 'radius':
radius}, ensure_ascii=False))

if __name__ == '__main__':
    main()

```

## E.2 .agents/skills/data-cleaning-and-visualization/scripts/robust\_loader.py

```

from __future__ import annotations

import argparse
import csv
import hashlib
import json
import sys
from datetime import datetime
from importlib import import_module
from pathlib import Path
from typing import Any

BASE_DIR = Path.cwd().resolve()
OUTPUT_DIR = BASE_DIR / "paper_output"
REPORT_FILE = OUTPUT_DIR / "data_cleaned" / "load_report.json"
INPUT_MANIFEST_FILE = OUTPUT_DIR / "input_manifest.json"

DATA_EXTS = {".xlsx", ".xls", ".csv", ".tsv", ".json"}
PDF_EXTS = {".pdf"}

def configure_utf8_stdio() -> None:
    for stream in (sys.stdout, sys.stderr):
        try:
            stream.reconfigure(encoding="utf-8")
        except Exception:
            pass

def safe_import(name: str):
    try:
        return import_module(name)
    except Exception:
        return None

def rel(path: Path) -> str:
    try:
        return path.resolve().relative_to(BASE_DIR).as_posix()
    except Exception:
        return str(path).replace("\\", "/")

def sha256_file(path: Path) -> str:
    if not path.exists() or not path.is_file():
        return ""
    digest = hashlib.sha256()
    with path.open("rb") as handle:

```

```

        for chunk in iter(lambda: handle.read(1024 * 1024), b''):
            digest.update(chunk)
    return digest.hexdigest()

def file_entry(path: Path, kind: str) -> dict[str, Any]:
    return {
        "path": rel(path),
        "kind": kind,
        "ext": path.suffix.lower(),
        "readable": False,
        "warnings": [],
        "errors": [],
    }

def inspect_xlsx(path: Path) -> dict[str, Any]:
    info = file_entry(path, "spreadsheet")
    openpyxl = safe_import("openpyxl")
    if openpyxl is None:
        info["errors"].append("缺少依赖 openpyxl, 无法取 .xlsx。")
        return info
    try:
        wb = openpyxl.load_workbook(path, read_only=True, data_only=True)
    except Exception as exc:
        info["errors"].append(f"无法打开 xlsx: {type(exc).__name__}: {exc}")
        return info
    sheets = []
    try:
        for sheet_name in wb.sheetnames:
            ws = wb[sheet_name]
            rows = ws.max_row or 0
            cols = ws.max_column or 0
            sample_cols: list[str] = []
            if rows and cols:
                first_row = next(ws.iter_rows(min_row=1, max_row=1, max_col=min(cols, 12),
values_only=True), None)
                if first_row:
                    sample_cols = [str(value) if value is not None else "" for value in
first_row]
            sheets.append({"name": sheet_name, "rows": rows, "cols": cols, "sample_cols":
sample_cols})
            if rows == 0 or cols == 0:
                info["warnings"].append(f"工作表 {sheet_name} 为空。")
    finally:
        wb.close()
    try:
        wb2 = openpyxl.load_workbook(path, read_only=False, data_only=True)
        merged_count = sum(len(wb2[name].merged_cells.ranges) for name in wb2.sheetnames)
        wb2.close()
        if merged_count:
            info["warnings"].append(f"检测到 {merged_count} 处合并单元格, 自动读取结果需人工核
对。")
    except Exception:
        pass
    info["readable"] = True
    info["sheets"] = sheets
    return info

def inspect_xls(path: Path) -> dict[str, Any]:
    info = file_entry(path, "spreadsheet")
    xlrd = safe_import("xlrd")
    if xlrd is None:
        info["errors"].append("缺少依赖 xlrd, 无法读取老 .xls; 建另存 .xlsx。")
        return info
    try:
        book = xlrd.open_workbook(str(path))
    except Exception as exc:
        info["errors"].append(f"无法打开 xls: {type(exc).__name__}: {exc}")

```

```

        return info
        info["sheets"] = [{"name": sheet.name, "rows": sheet.nrows, "cols": sheet.ncols,
"sample_cols": []} for sheet in book.sheets()]
        info["readable"] = True
        return info

def inspect_csv(path: Path) -> dict[str, Any]:
    info = file_entry(path, "table")
    pandas = safe_import("pandas")
    if pandas is not None:
        last_error: Exception | None = None
        for encoding in ("utf-8-sig", "utf-8", "gbk", "gb18030"):
            for sep in (None, ",", "\t", ";"):
                try:
                    df = pandas.read_csv(path, nrows=20, encoding=encoding, sep=sep,
engine="python")
                    info.update(
                        {
                            "readable": True,
                            "encoding": encoding,
                            "sep": sep if sep is not None else "auto",
                            "rows_sampled": int(len(df)),
                            "cols": int(len(df.columns)),
                            "sample_cols": [str(col) for col in df.columns.tolist()],
                        }
                    )
                    if df.empty:
                        info["warnings"].append("CSV 读取成功但样本为空。")
                        return info
                    except Exception as exc:
                        last_error = exc
                info["errors"].append(f"pandas 无法读取: {type(last_error).__name__}:
{last_error}")
            return info

        # Fallback without pandas.
        for encoding in ("utf-8-sig", "utf-8", "gbk", "gb18030"):
            try:
                with path.open("r", encoding=encoding, newline="") as handle:
                    sample = handle.read(4096)
                    dialect = csv.Sniffer().sniff(sample) if sample.strip() else csv.excel
                    handle.seek(0)
                    reader = csv.reader(handle, dialect)
                    first = next(reader, [])
                    info.update({"readable": True, "encoding": encoding, "sep": getattr(dialect,
"delimiter", ","), "sample_cols": [str(item) for item in first]})
                    return info
            except Exception:
                continue
            info["errors"].append("无法以常见编码读取 CSV/TSV。")
        return info

def inspect_json(path: Path) -> dict[str, Any]:
    info = file_entry(path, "json")
    for encoding in ("utf-8-sig", "utf-8", "gbk", "gb18030"):
        try:
            data = json.loads(path.read_text(encoding=encoding))
            info["readable"] = True
            info["encoding"] = encoding
            if isinstance(data, dict):
                info["top_level"] = "dict"
                info["sample_keys"] = list(data.keys())[:12]
            elif isinstance(data, list):
                info["top_level"] = "list"
                info["length"] = len(data)
                if data and isinstance(data[0], dict):
                    info["sample_keys"] = list(data[0].keys())[:12]
            else:

```



```

        info["top_level"] = type(data).__name__
    return info
except Exception:
    continue
info["errors"].append("JSON 无法以常见编码解析。")
return info

def inspect_pdf(path: Path) -> dict[str, Any]:
    info = file_entry(path, "pdf_diagnostic")
    info["readable"] = False
    pypdf = safe_import("pypdf")
    if pypdf is None:
        info["warnings"].append("缺少 pypdf, 无法断 PDF 文本。")
    else:
        try:
            reader = pypdf.PdfReader(str(path))
            total_pages = len(reader.pages)
            sample_pages = min(5, total_pages)
            char_count = 0
            for index in range(sample_pages):
                try:
                    char_count += len(reader.pages[index].extract_text() or "")
                except Exception:
                    pass
            info["pdf_pages"] = total_pages
            info["text_pages_sampled"] = sample_pages
            info["text_char_count"] = char_count
            if char_count == 0:
                info["warnings"].append("PDF 前几页未抽出文本, 可能是扫描版; 请 OCR 或人工转为 CSV/XLSX。")
            elif char_count < 200:
                info["warnings"].append("PDF 可抽文本很少, 请核对题面/表格是否完整。")
        except Exception as exc:
            info["warnings"].append(f"PDF 文本诊断失败: {type(exc).__name__}: {exc}")

    pdfplumber = safe_import("pdfplumber")
    if pdfplumber is None:
        info["table_diagnostic"] = {"available": False, "warning": "未安装 pdfplumber, 未做 PDF 表格诊断。"}
    else:
        try:
            table_count = 0
            table_samples: list[dict[str, Any]] = []
            with pdfplumber.open(path) as pdf:
                for page_index, page in enumerate(pdf.pages[:5], start=1):
                    tables = page.extract_tables() or []
                    table_count += len(tables)
                    for table in tables[:3]:
                        table_samples.append(
                            {
                                "page": page_index,
                                "rows": len(table),
                                "cols": max((len(row) for row in table), default=0),
                            }
                        )
            info["table_diagnostic"] = {
                "available": True,
                "table_count_first_5_pages": table_count,
                "samples": table_samples[:8],
                "warning": "PDF 表格抽取仅供诊断, 不能直接视为可信原始数据; 正式建模建议转 CSV/XLSX 后人工核对。",
            }
        except Exception as exc:
            info["table_diagnostic"] = {"available": False, "warning": f"PDF 表格断失: {type(exc).__name__}: {exc}"}
    return info

def iter_input_files(input_dirs: list[str]) -> list[Path]:

```

```

files: list[Path] = []
for text in input_dirs:
    path = (BASE_DIR / text).resolve()
    if path.exists() and path.is_dir():
        files.extend(item for item in path.rglob("*") if item.is_file())
return sorted(files, key=lambda item: item.as_posix().lower())

def load_input_manifest() -> dict[str, Any] | None:
    if not INPUT_MANIFEST_FILE.exists():
        return None
    try:
        data = json.loads(INPUT_MANIFEST_FILE.read_text(encoding="utf-8"))
    except Exception:
        return None
    return data if isinstance(data, dict) else None

def resolve_manifest_path(path_text: str) -> Path:
    path = Path(str(path_text or ""))
    if path.is_absolute():
        return path
    return BASE_DIR / path

def iter_manifest_raw_data(manifest: dict[str, Any]) -> tuple[list[Path], list[dict[str, Any]]]:
    files: list[Path] = []
    skipped: list[dict[str, Any]] = []
    entries = manifest.get("entries") if isinstance(manifest, dict) else []
    if not isinstance(entries, list):
        return files, skipped
    for entry in entries:
        if not isinstance(entry, dict):
            continue
        path_text = str(entry.get("path") or "")
        role = str(entry.get("role") or "")
        usable = bool(entry.get("usable_for_modeling"))
        path = resolve_manifest_path(path_text)
        if role == "raw_data" and usable:
            files.append(path)
        else:
            skipped.append(
                {
                    "path": path_text,
                    "role": role,
                    "reason": "not_raw_modeling_data" if role != "raw_data" else
                    "not_usable_for_modeling",
                }
            )
    return sorted(files, key=lambda item: item.as_posix().lower()), skipped

def inspect_file(path: Path) -> dict[str, Any] | None:
    ext = path.suffix.lower()
    if ext == ".xlsx":
        return inspect_xlsx(path)
    if ext == ".xls":
        return inspect_xls(path)
    if ext in {".csv", ".tsv"}:
        return inspect_csv(path)
    if ext == ".json":
        return inspect_json(path)
    if ext == ".pdf":
        return inspect_pdf(path)
    return None

def evaluate(input_dirs: list[str], use_manifest: bool = True) -> dict[str, Any]:
    input_manifest = load_input_manifest() if use_manifest else None

```

```

skipped_files: list[dict[str, Any]] = []
if input_manifest:
    files, skipped_files = iter_manifest_raw_data(input_manifest)
else:
    files = iter_input_files(input_dirs)
data_files: list[dict[str, Any]] = []
pdf_diagnostics: list[dict[str, Any]] = []
warnings: list[str] = []
errors: list[str] = []

if use_manifest and input_manifest is None and INPUT_MANIFEST_FILE.exists():
    warnings.append("input_manifest.json 存在但无法解析, 已回退为扫描 input_dirs。")
if input_manifest:
    warnings.extend(
        f"{item['path']}: 跳过 role={item['role']} ({item['reason']})。"
        for item in skipped_files
        if item.get("role") in {"result_template", "problem_statement",
"problem_statement_unreadable"}
    )

for path in files:
    info = inspect_file(path)
    if info is None:
        continue
    if path.suffix.lower() in PDF_EXTS:
        pdf_diagnostics.append(info)
    else:
        data_files.append(info)
    for item in info.get("warnings", []) or []:
        warnings.append(f"{info['path']}: {item}")
    for item in info.get("errors", []) or []:
        errors.append(f"{info['path']}: {item}")

readable_data = [item for item in data_files if item.get("readable")]
if not data_files:
    warnings.append("未发现 xlsx/xls/csv/tsv/json 数据文件; 若数据在 PDF 中, 请先人工核对并转为 CSV/XLSX。")
elif not readable_data:
    errors.append("发现数据文件但没有任何文件可读。")

return {
    "schema_version": "1.0",
    "generated_by": "data-cleaning-and-visualization/scripts/robust_loader.py",
    "generated_at": datetime.now().isoformat(timespec="seconds"),
    "status": "PASS" if not errors else "FAIL",
    "input_dirs": input_dirs,
    "input_manifest_used": bool(input_manifest),
    "input_manifest": rel(INPUT_MANIFEST_FILE) if input_manifest else "",
    "input_manifest_sha256": sha256_file(INPUT_MANIFEST_FILE) if input_manifest else "",
    "skipped_files": skipped_files,
    "data_files": data_files,
    "pdf_diagnostics": pdf_diagnostics,
    "summary": {
        "file_count_scanned": len(files),
        "data_file_count": len(data_files),
        "readable_data_file_count": len(readable_data),
        "pdf_file_count": len(pdf_diagnostics),
    },
    "warnings": warnings,
    "errors": errors,
}

def write_report(report: dict[str, Any]) -> None:
    REPORT_FILE.parent.mkdir(parents=True, exist_ok=True)
    REPORT_FILE.write_text(json.dumps(report, ensure_ascii=False, indent=2),
encoding="utf-8")

```

```

def main() -> int:
    configure_utf8_stdio()
    parser = argparse.ArgumentParser(description="Diagnose MathModel input data files and
write load_report.json.")
    parser.add_argument("--input-dir", action="append", dest="input_dirs", default=None,
help="Input directory to scan. Can be repeated.")
    args = parser.parse_args()
    input_dirs = args.input_dirs or ["problem_files", "crawled_data"]
    report = evaluate(input_dirs, use_manifest=args.input_dirs is None)
    write_report(report)
    print(f"load report: {rel(REPORT_FILE)}")
    if report["status"] == "PASS":
        print("[LOAD PASS]")
        for warning in report["warnings"][:8]:
            print(f" [warn] {warning}")
        return 0
    print("[LOAD FAIL]")
    for error in report["errors"][:12]:
        print(f" - {error}")
    return 1

if __name__ == "__main__":
    raise SystemExit(main())

```

### E.3 paper\_output/code/modeling/core\_solver.py

```

"""圆柱径向有限体积：后向欧拉、Picard 迭代和三对角求解。"""
from __future__ import annotations
import numba
import numpy as np

@numba.njit
def solve_thomas(a, b, c, d):
    n = len(d)
    upper = np.zeros(n)
    rhs = np.zeros(n)
    upper[0] = c[0] / b[0]
    rhs[0] = d[0] / b[0]
    for i in range(1, n):
        pivot = b[i] - a[i] * upper[i-1]
        upper[i] = c[i] / pivot
        rhs[i] = (d[i] - a[i] * rhs[i-1]) / pivot
    x = rhs.copy()
    for i in range(n-2, -1, -1):
        x[i] -= upper[i] * x[i+1]
    return x

@numba.njit
def interp_env(t, t_env, T_env, C_env):
    return np.interp(t, t_env, T_env), np.interp(t, t_env, C_env)

@numba.njit
def interp_radius(t, t_rad, R_rad):
    return np.interp(t, t_rad, R_rad)

@numba.njit
def properties(T, C, mode):
    if mode == 1:
        rho = np.full(len(C), 820.)
        cp = np.full(len(C), 2600.)
        k = np.full(len(C), .36)
        D = 7e-9 * np.exp(-.89 / C)
    elif mode == 2:
        rho = 650 + 128*C

```

```

        cp = 1450 + 2736*C/(1+C)
        k = .21 + .38*C/(1+C)
        D = 2.4e-3 * np.exp(-.45/C - 3850/T)
    else:
        rho = 760 + 90*C
        cp = 1850 + 2150*C/(1+C)
        k = .12 + .20*C/(1+C)
        D = 4.2e-4 * np.exp(-.30/C - 3850/T)
    return rho*cp, k, D

@numba.njit
def system(coef, storage, old, weights, R, dt, transfer, exterior):
    # 用xi 环面积权重；内部通量成对抵消，表面采用半格串联阻力。
    n = len(old)
    dxi = 1. / n
    face = 2*coef[:-1]*coef[1:]/(coef[:-1]+coef[1:])
    g = 2*np.arange(1,n)*face/(R*R)
    a = np.zeros(n)
    c = np.zeros(n)
    a[1:] = -g
    c[:-1] = -g
    mass = weights*storage/dt
    b = mass-a-c
    rhs = mass*old
    gamma = 1/(1/transfer + .5*dxi*R/coef[-1])
    b[-1] += 2*gamma/R
    rhs[-1] += 2*gamma/R*exterior
    return a,b,c,rhs,gamma

@numba.njit
def residual(a,b,c,rhs,x):
    r = b*x-rhs
    scale = np.abs(b*x)+np.abs(rhs)
    r[1:] += a[1:]*x[:-1]
    r[:-1] += c[:-1]*x[1:]
    scale[1:] += np.abs(a[1:]*x[:-1])
    scale[:-1] += np.abs(c[:-1]*x[1:])
    return np.max(np.abs(r))/np.maximum(scale,1e-30))

@numba.njit
def _simulate(t_env,T_env,C_env,t_rad,R_rad,total_seconds,formula_mode,
             sample_every_s,N,h,hm,dt,stop_at_cmax,max_iterations,converge,
             initial_T,initial_C):
    edges = np.linspace(0.,1.,N+1)
    weights = edges[1:]**2-edges[:-1]**2
    one = np.ones(N)
    T = np.full(N,initial_T)
    C = np.full(N,initial_C)
    capacity = int(np.floor(total_seconds/sample_every_s))+2
    times = np.zeros(capacity)
    radii = np.zeros(capacity)
    Th = np.zeros((capacity,N))
    Ch = np.zeros((capacity,N))
    radii[0] = R_rad[0]
    Th[0],Ch[0] = T,C
    rec = 1
    sample_index = 1
    t = 0.
    steps = 0
    iterations_sum = 0
    max_it = 0
    flux_sum = 0.
    balance_max = 0.
    step_balance_max = 0.
    residual_C_max = 0.
    residual_T_max = 0.
    delta_C_max = 0.

```

```

delta_T_max = 0.
center_excess_max = 0.
left_t = 0.
left_C = initial_C
status = 0
while t < total_seconds-1e-10:
    step_dt = min(dt,total_seconds-t)
    t_new = t+step_dt
    R = interp_radius(t_new,t_rad,R_rad)
    air,Ce = interp_env(t_new,t_env,T_env,C_env)
    T_old,C_old = T.copy(),C.copy()
    Ti,Ci = T.copy(),C.copy()
    accepted = False
    rc,rt,dc,dT = 0.,0.,0.,0.
    for it in range(max_iterations):
        storage,k,D = properties(Ti,Ci,formula_mode)
        ac,bc,cc,fc,gamma = system(D,one,C_old,weights,R,step_dt,hm,Ce)
        at,bt,ct,ft,_ = system(k,storage,T_old,weights,R,step_dt,h,air)
        Cn = solve_thomas(ac,bc,cc,fc)
        Tn = solve_thomas(at,bt,ct,ft)
        dc = np.max(np.abs(Cn-Ci))
        dT = np.max(np.abs(Tn-Ti))
        Ci,Ti = Cn,Tn
        # 接近收敛时，重组最终状态的非线性方程并检查残差。
        if (dc <= 1e-10 and dT <= 1e-7) or it == max_iterations-1:
            sn,kn,Dn = properties(Ti,Ci,formula_mode)
            aa,bb,ccn,ff,_ = system(Dn,one,C_old,weights,R,step_dt,hm,Ce)
            rc = residual(aa,bb,ccn,ff,Ci)
            aa,bb,ccn,ff,_ = system(kn,sn,T_old,weights,R,step_dt,h,air)
            rt = residual(aa,bb,ccn,ff,Ti)
            if not converge or (dc<=1e-10 and dT<=1e-7 and max(rc,rt)<=1e-10):
                accepted = True
                break
        if not accepted or not np.isfinite(Ci).all() or not np.isfinite(Ti).all() or
np.min(Ci)<=0:
            status = -1
            break
        iterations_sum += it+1
        max_it = max(max_it,it+1)
        residual_C_max = max(residual_C_max,rc)
        residual_T_max = max(residual_T_max,rt)
        delta_C_max = max(delta_C_max,dc)
        delta_T_max = max(delta_T_max,dT)
        # gamma 为本步实际水分矩阵采用的系数，不用重组系数替代它。
        outward = 2*gamma/R*(Ci[-1]-Ce)*step_dt
        flux_sum += outward
        mean_new = np.sum(weights*Ci)
        step_balance_max =
max(step_balance_max,abs(np.sum(weights*(Ci-C_old))+outward)/initial_C)
        balance_max = max(balance_max,abs(mean_new-initial_C+flux_sum)/initial_C)
        center_excess_max = max(center_excess_max,(Ci[0]-Ci[1])/8)
        # 输出不改变积分步长。跨越输出点时作线性输出插值；本题验证节点整除步长。
        while sample_index*sample_every_s <= t_new+1e-10:
            if rec >= capacity:
                raise IndexError('output capacity')
            ts = sample_index*sample_every_s
            alpha = (ts-t)/step_dt
            times[rec] = ts
            radii[rec] = interp_radius(ts,t_rad,R_rad)
            Th[rec] = T_old+alpha*(Ti-T_old)
            Ch[rec] = C_old+alpha*(Ci-C_old)
            rec += 1
            sample_index += 1
        left_t,left_C = t,np.max(C_old)
        t,T,C = t_new,Ti,Ci
        steps += 1
        if stop_at_cmax > 0 and np.max(C) < stop_at_cmax:
            status = 1
            break
    if abs(times[rec-1]-t)>1e-9:

```

```

        times[rec] = t
        radii[rec] = interp_radius(t,t_rad,R_rad)
        Th[rec],Ch[rec] = T,C
        rec += 1
    diag = np.array([status,left_t,t,left_C,np.max(C),balance_max,step_balance_max,
                    residual_C_max,residual_T_max,max_it,iterations_sum/max(steps,1),
                    steps,center_excess_max,delta_C_max,delta_T_max])
    return times[:rec],radii[:rec],Th[:rec],Ch[:rec],t,rec,diag

def _finish(result, moving, return_diagnostics):
    keys = ['status_code','left_time_s','end_time_s','left_max_C','final_max_C',
            'water_balance','step_water_balance','residual_C','residual_T',
            'max_iterations','mean_iterations','steps','center_excess_bound',
            'last_iteration_delta_C_max','last_iteration_delta_T_max']
    diag = dict(zip(keys,result[-1].tolist()))
    diag['status'] = {-1:'solver_failed',0:'not_reached',1:'reached'}
    [int(diag['status_code'])]
    values = result[:-1] if moving else
    (result[0],result[2],result[3],result[4],result[5])
    if return_diagnostics:
        return (*values,diag)
    if diag['status_code'] == -1:
        raise RuntimeError(f"Nonlinear solve failed after t={result[4]} s")
    return values

def simulate_fvm_fixed(t_env,T_env,C_env,total_seconds,formula_mode,sample_every_s=1,
                      N=80,R=.02,h=25.,hm=8e-7,dt=1.,stop_at_cmax=-1.,
                      max_iterations=50,converge=True,return_diagnostics=False,
                      initial_T=301.15,initial_C=2.55):
    result = _simulate(t_env,T_env,C_env,np.array([0.,float(total_seconds)]),
                      np.array([R,R]),total_seconds,formula_mode,sample_every_s,N,h,hm,dt,
                      stop_at_cmax,max_iterations,converge,initial_T,initial_C)
    return _finish(result,False,return_diagnostics)

def simulate_fvm_moving(t_env,T_env,C_env,t_rad,R_rad,total_seconds,sample_every_s=60,
                      N=80,h=25.,hm=8e-7,dt=1.,stop_at_cmax=.15,
                      max_iterations=50,converge=True,return_diagnostics=False,
                      initial_T=301.15,initial_C=2.55):
    result =
    _simulate(t_env,T_env,C_env,t_rad,R_rad,total_seconds,3,sample_every_s,N,h,hm,
             dt,stop_at_cmax,max_iterations,converge,initial_T,initial_C)
    return _finish(result,True,return_diagnostics)

def reconstruct_surface(
    times: np.ndarray,
    T_outer: np.ndarray,
    C_outer: np.ndarray,
    delta: float | np.ndarray,
    t_env: np.ndarray,
    T_env: np.ndarray,
    C_env: np.ndarray,
    formula_mode: int,
    h: float,
    hm: float,
) -> tuple[np.ndarray, np.ndarray]:
    if formula_mode == 1:
        k = 0.36
        D = 7.0e-9 * np.exp(-0.89 / C_outer)
    elif formula_mode == 2:
        k = 0.21 + 0.38 * (C_outer / (C_outer + 1.0))
        D = 2.4e-3 * np.exp(-0.45 / C_outer - 3850.0 / T_outer)
    else: # Appendix 4
        k = 0.12 + 0.20 * (C_outer / (C_outer + 1.0))
        D = 4.2e-4 * np.exp(-0.30 / C_outer - 3850.0 / T_outer)

    T_air = np.interp(times, t_env, T_env)

```

```

C_air = np.interp(times, t_env, C_env)
# Match the half-cell resistance used by the Robin boundary.
T_surface = (k * T_outer + h * delta * T_air) / (k + h * delta)
C_surface = (D * C_outer + hm * delta * C_air) / (D + hm * delta)
# The initial condition specifies a uniform field, including the surface.
T_surface[times == 0.0] = T_outer[times == 0.0]
C_surface[times == 0.0] = C_outer[times == 0.0]
return T_surface, C_surface

def sample_and_interpolate_fixed(
    times_raw: np.ndarray,
    T_raw: np.ndarray,
    C_raw: np.ndarray,
    N: int,
    R: float,
    r_target_cm: np.ndarray,
    t_env: np.ndarray,
    T_env: np.ndarray,
    C_env: np.ndarray,
    formula_mode: int,
    h: float = 25.0,
    hm: float = 8e-7,
) -> tuple[np.ndarray, np.ndarray]:
    r_faces = np.linspace(0.0, R, N + 1)
    r_centers = 0.5 * (r_faces[:-1] + r_faces[1:])
    r_target_m = r_target_cm / 100.0
    n_steps = len(times_raw)
    n_radii = len(r_target_m)
    T_interp = np.zeros((n_steps, n_radii), dtype=np.float64)
    C_interp = np.zeros((n_steps, n_radii), dtype=np.float64)
    T_surface, C_surface = reconstruct_surface(
        times_raw, T_raw[:, -1], C_raw[:, -1], R / (2 * N),
        t_env, T_env, C_env, formula_mode, h, hm,
    )
    r_nodes = np.append(r_centers, R)
    for s in range(n_steps):
        T_interp[s] = np.interp(r_target_m, r_nodes, np.append(T_raw[s], T_surface[s]))
        C_interp[s] = np.interp(r_target_m, r_nodes, np.append(C_raw[s], C_surface[s]))
    return T_interp, C_interp

def sample_and_interpolate_moving(
    times_raw: np.ndarray,
    radius_raw: np.ndarray,
    T_raw: np.ndarray,
    C_raw: np.ndarray,
    N: int,
    r_target_cm: np.ndarray,
    t_env: np.ndarray,
    T_env: np.ndarray,
    C_env: np.ndarray,
    h: float = 25.0,
    hm: float = 8e-7,
) -> tuple[np.ndarray, np.ndarray]:
    xi_faces = np.linspace(0.0, 1.0, N + 1)
    xi_centers = 0.5 * (xi_faces[:-1] + xi_faces[1:])
    r_target_m = r_target_cm / 100.0
    n_steps = len(times_raw)
    n_radii = len(r_target_m)
    C_interp = np.full((n_steps, n_radii), np.nan, dtype=np.float64)
    _, C_surface = reconstruct_surface(
        times_raw, T_raw[:, -1], C_raw[:, -1], radius_raw / (2 * N),
        t_env, T_env, C_env, 3, h, hm,
    )
    xi_nodes = np.append(xi_centers, 1.0)
    for s in range(n_steps):
        R_s = radius_raw[s]
        valid_mask = r_target_m <= R_s
        if np.any(valid_mask):

```



```

        xi_targets = r_target_m[valid_mask] / R_s
        C_interp[s, valid_mask] = np.interp(xi_targets, xi_nodes, np.append(C_raw[s],
C_surface[s]))
    return C_interp, C_surface

```

## E.4 paper\_output/code/modeling/q1\_model.py

```

# 问题 1：使用题给数据求解并导出题目要求的结果。
from __future__ import annotations

import csv
import json
from pathlib import Path
import numpy as np
import pandas as pd
import openpyxl

from core_solver import simulate_fvm_fixed, sample_and_interpolate_fixed
from result_contract_io import (
    upsert_question_contracts,
    solver_metrics,
    write_csv_table,
    table_entry,
    round_float,
    PROJECT_ROOT,
    OUTPUT_DIR,
    TABLES_DIR,
    RESULTS_DIR,
    DATA_CLEANED_DIR,
)

QUESTION = {
    'question_id': 'AQ1',
    'title': '平衡段温度与干基含水率',
    'task_type': '非线性扩散与热传导',
    'main_model': '隐式有限体积法 (FVM) + Picard 迭代 + Thomas 三对角求解',
}

def run_aq1():
    config=json.loads((OUTPUT_DIR/'qa/model_validation_checks.json').read_text(encoding='utf-8'
    '))[ 'selected_configuration' ]
    N_run,dt_run=config['N'],config['dt_s']
    env_path = DATA_CLEANED_DIR / 'a_environment.csv'
    df_env = pd.read_csv(env_path)
    t_env = df_env['time_s'].values.astype(np.float64)
    T_env = df_env['temperature_K'].values.astype(np.float64)
    C_env = df_env['air_moisture_kgkg'].values.astype(np.float64)

    # 1. Run simulation for 1800 s with formula_mode=1 (Appendix 2)
    times_raw, T_raw, C_raw, end_time, n_rec, diagnostics = simulate_fvm_fixed(
        t_env=t_env,
        T_env=T_env,
        C_env=C_env,
        total_seconds=1800,
        formula_mode=1,
        sample_every_s=1,
        N=N_run,
        R=0.02,
        h=25.0,
        hm=8e-7,
        dt=dt_run,
        return_diagnostics=True,
    )

    if diagnostics['status']=='solver_failed':
        raise RuntimeError('AQ1 nonlinear solve failed')

    # 2. Build Table 1 (Temperature) & Table 2 (Moisture)

```

```

times_tab = np.array([100, 300, 600, 900, 1200, 1500, 1800], dtype=np.float64)
r_tab_cm = np.array([0.0, 0.5, 1.0, 1.5, 2.0], dtype=np.float64)

# Interpolate to table points
T_tab, C_tab = sample_and_interpolate_fixed(times_tab, T_raw[times_tab.astype(int)],
C_raw[times_tab.astype(int)], N=N_run, R=0.02, r_target_cm=r_tab_cm, t_env=t_env,
T_env=T_env, C_env=C_env, formula_mode=1)

# Convert Temperature to Celsius
T_tab_C = T_tab - 273.15

# Write Table 1 CSV
table1_rows = []
for i, t_val in enumerate(times_tab):
    row = {'时间/s': int(t_val)}
    for j, r_val in enumerate(r_tab_cm):
        row[f'{r_val:g} cm'] = f'{T_tab_C[i, j]:.4f}'
    table1_rows.append(row)
t1_path = TABLES_DIR / 'table_aq1_temperature.csv'
write_csv_table(t1_path, table1_rows)

# Write Table 2 CSV
table2_rows = []
for i, t_val in enumerate(times_tab):
    row = {'时间/s': int(t_val)}
    for j, r_val in enumerate(r_tab_cm):
        row[f'{r_val:g} cm'] = f'{C_tab[i, j]:.4f}'
    table2_rows.append(row)
t2_path = TABLES_DIR / 'table_aq1_moisture.csv'
write_csv_table(t2_path, table2_rows)

# 3. Build result1.xlsx (1800 s, 0.1 cm steps)
r_full_cm = np.arange(0.0, 2.0001, 0.1)
T_full, C_full = sample_and_interpolate_fixed(times_raw, T_raw, C_raw, N=N_run,
R=0.02, r_target_cm=r_full_cm, t_env=t_env, T_env=T_env, C_env=C_env, formula_mode=1)
T_full_C = T_full - 273.15

res1_path = TABLES_DIR / 'result1.xlsx'
wb = openpyxl.Workbook(write_only=True)
ws_T = wb.create_sheet('温度')
ws_C = wb.create_sheet('水分浓度')

header = ['时间\\到药材中心的距离'] + [round(r, 1) for r in r_full_cm]
ws_T.append(header)
ws_C.append(header)

# 1 to 1800 s
for s in range(1, 1801):
    row_T = [float(times_raw[s])] + [round(val, 4) for val in T_full_C[s]]
    row_C = [float(times_raw[s])] + [round(val, 4) for val in C_full[s]]
    ws_T.append(row_T)
    ws_C.append(row_C)
wb.save(res1_path)

# 4. Metrics & Contracts
r_faces = np.linspace(0.0, 0.02, N_run+1)
vols = np.pi * (r_faces[1:]**2 - r_faces[:-1]**2)
C_mean = np.sum(vols * C_raw[-1]) / np.sum(vols)
metrics = [
    {'metric_name': 'aq1_final_center_temperature_C', 'metric_role': 'evaluation',
'value': round_float(T_full_C[1800, 0], 4), 'unit': '°C'},
    {'metric_name': 'aq1_final_surface_temperature_C', 'metric_role': 'evaluation',
'value': round_float(T_full_C[1800, -1], 4), 'unit': '°C'},
    {'metric_name': 'aq1_final_center_moisture', 'metric_role': 'evaluation', 'value':
round_float(C_full[1800, 0], 4), 'unit': 'kg/kg'},
    {'metric_name': 'aq1_final_surface_moisture', 'metric_role': 'evaluation',
'value': round_float(C_full[1800, -1], 4), 'unit': 'kg/kg'},
    {'metric_name': 'aq1_total_moisture_loss_fraction', 'metric_role': 'evaluation',
'value': round_float((2.55 - C_mean) / 2.55, 6), 'unit': 'fraction'},
]

```

```

metrics += solver_metrics(diagnostics, 'AQ1')

conclusions = [
    {
        'question_id': 'AQ1',
        'conclusion_text': f'在预热平衡阶段 (0-1800 s), 赫受烘房升温与流麴中心
温度由 28.0000°C 上升至{T_full_C[1800, 0]:.4f}°C, 表面温度达到{T_full_C[1800, -1]:.4f}°C; 由于
对流传质阻力与内部非线性水分扩散制约, 药材表面干基含水率由 2.5500 kg/kg 下降至{C_full[1800,
-1]:.4f} kg/kg, 中心干基含水率为{C_full[1800, 0]:.4f} kg/kg, 体现出显著的内外梯度迟滞效应。',
        'evidence_required': ['model_results.json', 'metrics.json',
'table_index.json'],
        'evidence_status': 'computed',
    }
]

tables = [
    table_entry(
        question_id='AQ1',
        table_id='table1',
        title='30 分钟内药材的温度 (表 1) ',
        purpose='展示预热阶段 100-1800 s 各径向截面的药材温度动态。',
        path=t1_path,
        status='computed',
    ),
    table_entry(
        question_id='AQ1',
        table_id='table2',
        title='30 分钟内药材的水分浓度 (表 2) ',
        purpose='展示预热阶段 100-1800 s 各径向截面的干基水分浓度分布。',
        path=t2_path,
        status='computed',
    ),
]

outputs = [
    {'path': str(t1_path), 'type': 'csv'},
    {'path': str(t2_path), 'type': 'csv'},
    {'path': str(res1_path), 'type': 'xlsx'},
]

summary = f'AQ1 预热平衡阶段数值求解完成: 1800 s 时中心温度为{T_full_C[1800, 0]:.4f}°C, 表面
温度为{T_full_C[1800, -1]:.4f}°C; 中心水分浓度为{C_full[1800, 0]:.4f} kg/kg, 表面水分浓度为
{C_full[1800, -1]:.4f} kg/kg。表 1、表 2 及 result1.xlsx 均已导出。'

upsert_question_contracts(
    question=QUESTION,
    result_summary=summary,
    metrics=metrics,
    tables=tables,
    conclusions=conclusions,
    outputs=outputs,
    status='computed',
    parameters=[{'name': 'N', 'value': N_run}, {'name': 'dt_s', 'value': dt_run},
{'name': 'event_status', 'value': diagnostics['status']}],
)
print('AQ1 finished successfully.')

if __name__ == '__main__':
    run_aq1()

```

## E.5 paper\_output/code/modeling/q2\_model.py

```

# 问题 2: 使用题给数据求解并导出题目要求的结果。
from __future__ import annotations

import csv
import json
from pathlib import Path

```

```

import numpy as np
import pandas as pd
import openpyxl

from core_solver import simulate_fvm_fixed, sample_and_interpolate_fixed
from result_contract_io import (
    upsert_question_contracts,
    solver_metrics,
    write_csv_table,
    table_entry,
    round_float,
    PROJECT_ROOT,
    OUTPUT_DIR,
    TABLES_DIR,
    RESULTS_DIR,
    DATA_CLEANED_DIR,
)

QUESTION = {
    'question_id': 'AQ2',
    'title': '统一经验物性的全过程模型',
    'task_type': '热湿非线性耦合',
    'main_model': '附录3 全过程经验物性FVM模型 + Picard迭代 + Thomas三对角求解',
}

def run_aq2():
    config=json.loads((OUTPUT_DIR/'qa/model_validation_checks.json').read_text(encoding='utf-8'))['selected_configuration']
    N_run,dt_run=config['N'],config['dt_s']
    env_path = DATA_CLEANED_DIR / 'a_environment.csv'
    df_env = pd.read_csv(env_path)
    t_env = df_env['time_s'].values.astype(np.float64)
    T_env = df_env['temperature_K'].values.astype(np.float64)
    C_env = df_env['air_moisture_kgkg'].values.astype(np.float64)

    # Appendix 3 applies from t=0 until drying completes. Tables 3/4
    # show only the first 3 h; result2.xlsx must retain every second.
    times_raw, T_raw, C_raw, end_time, n_rec, diagnostics = simulate_fvm_fixed(
        t_env=t_env,
        T_env=T_env,
        C_env=C_env,
        total_seconds=259200,
        formula_mode=2,
        sample_every_s=1,
        N=N_run,
        R=0.02,
        h=25.0,
        hm=8e-7,
        dt=dt_run,
        stop_at_cmax=0.15,
        return_diagnostics=True,
    )
    if diagnostics['status'] != 'reached':
        raise RuntimeError(diagnostics['status'])

    # 2. Build Table 3 (Temperature) & Table 4 (Moisture)
    # 0.5 to 3.0 h every 0.5 h -> [1800, 3600, 5400, 7200, 9000, 10800] s
    times_tab_s = np.array([1800, 3600, 5400, 7200, 9000, 10800], dtype=np.float64)
    times_tab_h = times_tab_s / 3600.0
    r_tab_cm = np.array([0.0, 0.5, 1.0, 1.5, 2.0], dtype=np.float64)

    T_tab, C_tab = sample_and_interpolate_fixed(times_tab_s,
        T_raw[times_tab_s.astype(int)], C_raw[times_tab_s.astype(int)], N=N_run, R=0.02,
        r_target_cm=r_tab_cm, t_env=t_env, T_env=T_env, C_env=C_env, formula_mode=2)
    T_tab_C = T_tab - 273.15

    # Write Table 3 CSV
    table3_rows = []

```

```

for i, t_h in enumerate(times_tab_h):
    row = {'时间/h': f'{t_h:.1f}'}
    for j, r_val in enumerate(r_tab_cm):
        row[f'{r_val:g} cm'] = f'{T_tab_C[i, j]:.4f}'
    table3_rows.append(row)
t3_path = TABLES_DIR / 'table_aq2_temperature.csv'
write_csv_table(t3_path, table3_rows)

# Write Table 4 CSV
table4_rows = []
for i, t_h in enumerate(times_tab_h):
    row = {'时间/h': f'{t_h:.1f}'}
    for j, r_val in enumerate(r_tab_cm):
        row[f'{r_val:g} cm'] = f'{C_tab[i, j]:.4f}'
    table4_rows.append(row)
t4_path = TABLES_DIR / 'table_aq2_moisture.csv'
write_csv_table(t4_path, table4_rows)

# 3. Build result2.xlsx (every second to the terminal event, 0.1 cm steps)
r_full_cm = np.arange(0.0, 2.0001, 0.1)
T_full, C_full = sample_and_interpolate_fixed(times_raw, T_raw, C_raw, N=N_run,
R=0.02, r_target_cm=r_full_cm, t_env=t_env, T_env=T_env, C_env=C_env, formula_mode=2)
T_full_C = T_full - 273.15

res2_path = TABLES_DIR / 'result2.xlsx'
wb = openpyxl.Workbook(write_only=True)
ws_T = wb.create_sheet('温度')
ws_C = wb.create_sheet('水分浓度')

header = ['时间\\到药材中心的距离'] + [round(r, 1) for r in r_full_cm]
ws_T.append(header)
ws_C.append(header)

for s in range(1, len(times_raw)):
    row_T = [float(times_raw[s])] + [round(val, 4) for val in T_full_C[s]]
    row_C = [float(times_raw[s])] + [round(val, 4) for val in C_full[s]]
    ws_T.append(row_T)
    ws_C.append(row_C)
wb.save(res2_path)

# 4. Metrics & Contracts
metrics = [
    {'metric_name': 'aq2_3h_center_temperature_C', 'metric_role': 'evaluation',
'value': round_float(T_full_C[10800, 0], 4), 'unit': '°C'},
    {'metric_name': 'aq2_3h_surface_temperature_C', 'metric_role': 'evaluation',
'value': round_float(T_full_C[10800, -1], 4), 'unit': '°C'},
    {'metric_name': 'aq2_3h_center_moisture', 'metric_role': 'evaluation', 'value':
round_float(C_full[10800, 0], 4), 'unit': 'kg/kg'},
    {'metric_name': 'aq2_3h_surface_moisture', 'metric_role': 'evaluation', 'value':
round_float(C_full[10800, -1], 4), 'unit': 'kg/kg'},
    {'metric_name': 'aq2_1800s_surface_temperature_C', 'metric_role': 'comparison',
'value': round_float(T_full_C[1800, -1], 4), 'unit': '°C'},
    {'metric_name': 'aq2_1800s_surface_moisture', 'metric_role': 'comparison',
'value': round_float(C_full[1800, -1], 4), 'unit': 'kg/kg'},
]

metrics += solver_metrics(diagnostics, 'AQ2')

conclusions = [
    {
        'question_id': 'AQ2',
        'conclusion_text': f'采用附录3全过程统一经验物性公式自 t=0 连续求解 3 h (10800 s) :
材升温迅速并在 3 h 内达到中心{T_full_C[10800, 0]:.4f}°C、表面{T_full_C[10800, -1]:.4f}°C，整体
近烘房温度（约 50.2°C）；水分浓度在 3 h 末中心降至{C_full[10800, 0]:.4f} kg/kg，表面降至
{C_full[10800, -1]:.4f} kg/kg。相较于 AQ1 常物性模型，附录3通过温度和含水率相关物性描述全过程，不
需要拼接问题一末态。',
        'evidence_required': ['model_results.json', 'metrics.json',
'table_index.json'],
        'evidence_status': 'computed',
    }
]

```

```

]

tables = [
    table_entry(
        question_id='AQ2',
        table_id='table3',
        title='3 小时内药材的温度 (表 3) ',
        purpose='展示全过程模型在 0.5-3.0 h 各半小时的温度径向分布。',
        path=t3_path,
        status='computed',
    ),
    table_entry(
        question_id='AQ2',
        table_id='table4',
        title='3 小时内药材的水分浓度 (表 4) ',
        purpose='展示全过程模型在 0.5-3.0 h 各半小时的干基水分浓度径向分布。',
        path=t4_path,
        status='computed',
    ),
]

outputs = [
    {'path': str(t3_path), 'type': 'csv'},
    {'path': str(t4_path), 'type': 'csv'},
    {'path': str(res2_path), 'type': 'xlsx'},
]

summary = f'AQ2 全过程模型求解至达标时刻{end_time:.1f} s : 3 h 时中心温度为{T_full_C[10800, 0]:.4f}°C, 表面温度为{T_full_C[10800, -1]:.4f}°C ; 中心水分浓度为{C_full[10800, 0]:.4f} kg/kg, 表面水分浓度为{C_full[10800, -1]:.4f} kg/kg 。表 3、表 4 保留前 3 h, result2.xlsx 包含 1 s 至达标时刻的逐秒温度和含水率。'

upsert_question_contracts(
    question=QUESTION,
    result_summary=summary,
    metrics=metrics,
    tables=tables,
    conclusions=conclusions,
    outputs=outputs,
    status='computed',
    parameters=[{'name': 'N', 'value': N_run}, {'name': 'dt_s', 'value': dt_run},
{'name': 'event_status', 'value': diagnostics['status']}],
)
print('AQ2 finished successfully.')

if __name__ == '__main__':
    run_aq2()

```

## E.6 paper\_output/code/modeling/q3\_model.py

```

# 图 3 : 使用题给数据求解并导出题目要求的结果。
from __future__ import annotations

import csv
import json
from pathlib import Path
import numpy as np
import pandas as pd
import openpyxl

from core_solver import simulate_fvm_fixed, sample_and_interpolate_fixed
from result_contract_io import (
    upsert_question_contracts,
    solver_metrics,
    write_csv_table,
    table_entry,
    round_float,
    PROJECT_ROOT,
    OUTPUT_DIR,
)

```

```

    TABLES_DIR,
    RESULTS_DIR,
    DATA_CLEANED_DIR,
)

QUESTION = {
    'question_id': 'AQ3',
    'title': '固定半径下全域达标时长',
    'task_type': '阈值事件检测',
    'main_model': '烘干终点全域极值检测 + 离散阈值事件检测',
}

def run_aq3():

    config=json.loads((OUTPUT_DIR/'qa/model_validation_checks.json').read_text(encoding='utf-8'
    '))[ 'selected_configuration']
    N_run,dt_run=config['N'],config['dt_s']
    env_path = DATA_CLEANED_DIR / 'a_environment.csv'
    df_env = pd.read_csv(env_path)
    t_env = df_env['time_s'].values.astype(np.float64)
    T_env = df_env['temperature_K'].values.astype(np.float64)
    C_env = df_env['air_moisture_kgkg'].values.astype(np.float64)

    # 1. Run simulation until max(C) < 0.15 kg/kg (sampling every 60 s)
    # Maximum horizon 72 h = 259200 s
    times_raw, T_raw, C_raw, end_time, n_rec, diagnostics = simulate_fvm_fixed(
        t_env=t_env,
        T_env=T_env,
        C_env=C_env,
        total_seconds=259200,
        formula_mode=2,
        sample_every_s=60,
        N=N_run,
        R=0.02,
        h=25.0,
        hm=8e-7,
        dt=dt_run,
        stop_at_cmax=0.15,
        return_diagnostics=True,
    )
    if diagnostics['status'] != 'reached':
        raise RuntimeError(diagnostics['status'])

    t_end_s = end_time
    t_end_h = t_end_s / 3600.0

    # 2. Build Table 5 (Moisture every 6 h + drying end time)
    # Target radii: 0, 0.5, 1.0, 1.5, 2.0 cm
    r_tab_cm = np.array([0.0, 0.5, 1.0, 1.5, 2.0], dtype=np.float64)

    # 6 h intervals in seconds: 6h=21600s, 12h=43200s, ... up to t_end_s
    max_6h = int(t_end_s // 21600)
    times_6h_s = [i * 21600 for i in range(1, max_6h + 1)]
    if t_end_s not in times_6h_s:
        times_tab_s = np.array(times_6h_s + [t_end_s], dtype=np.float64)
    else:
        times_tab_s = np.array(times_6h_s, dtype=np.float64)

    # Find the matching indices in times_raw
    tab_indices = []
    for ts in times_tab_s:
        idx = np.argmin(np.abs(times_raw - ts))
        tab_indices.append(idx)
    tab_indices = np.array(tab_indices)

    T_tab, C_tab = sample_and_interpolate_fixed(
        times_tab_s,
        T_raw[tab_indices],
        C_raw[tab_indices],

```

```

        N=N_run,
        R=0.02,
        r_target_cm=r_tab_cm,
        t_env=t_env, T_env=T_env, C_env=C_env, formula_mode=2,
    )

    table5_rows = []
    for i, ts in enumerate(times_tab_s):
        if i == len(times_tab_s) - 1 and abs(ts - t_end_s) < 1e-3:
            time_label = f'烘干结束时间 ({t_end_h:.4f} h)'
        else:
            time_label = f'{ts / 3600.0:.1f}'
        row = {'时间/h': time_label}
        for j, r_val in enumerate(r_tab_cm):
            row[f'{r_val:g} cm'] = f'{C_tab[i, j]:.4f}'
        table5_rows.append(row)

    t5_path = TABLES_DIR / 'table_aq3_drying_moisture.csv'
    write_csv_table(t5_path, table5_rows)

    # 3. Build result3.xlsx (every 60 s + end time, 0.1 cm steps)
    r_full_cm = np.arange(0.0, 2.0001, 0.1)
    T_full, C_full = sample_and_interpolate_fixed(
        times_raw,
        T_raw,
        C_raw,
        N=N_run,
        R=0.02,
        r_target_cm=r_full_cm,
        t_env=t_env, T_env=T_env, C_env=C_env, formula_mode=2,
    )

    res3_path = TABLES_DIR / 'result3.xlsx'
    wb = openpyxl.Workbook(write_only=True)
    ws = wb.create_sheet('Sheet1')

    header = ['时间\\到药材中心的距离'] + [round(r, 1) for r in r_full_cm]
    ws.append(header)

    for i in range(1, len(times_raw)):
        t_sec = float(times_raw[i])
        row = [t_sec] + [round(val, 4) for val in C_full[i]]
        ws.append(row)
    wb.save(res3_path)

    # 4. Metrics & Contracts
    metrics = [
        {'metric_name': 'aq3_drying_time_s', 'metric_role': 'evaluation', 'value':
        round_float(t_end_s, 2), 'unit': 's'},
        {'metric_name': 'aq3_drying_time_h', 'metric_role': 'evaluation', 'value':
        round_float(t_end_h, 4), 'unit': 'h'},
        {'metric_name': 'aq3_final_max_moisture', 'metric_role': 'evaluation', 'value':
        round_float(np.max(C_full[-1]), 4), 'unit': 'kg/kg'},
        {'metric_name': 'aq3_final_min_moisture', 'metric_role': 'evaluation', 'value':
        round_float(np.min(C_full[-1]), 4), 'unit': 'kg/kg'},
        {'metric_name': 'aq3_final_center_moisture', 'metric_role': 'evaluation', 'value':
        round_float(C_full[-1, 0], 4), 'unit': 'kg/kg'},
        {'metric_name': 'aq3_final_surface_moisture', 'metric_role': 'evaluation',
        'value': round_float(C_full[-1, -1], 4), 'unit': 'kg/kg'},
    ]

    metrics += solver_metrics(diagnostics, 'AQ3')

    conclusions = [
        {
            'question_id': 'AQ3',
            'conclusion_text': f'在固定几何尺寸（半径 2 cm）下，严格以药材全域最大含水率低于 0.15
            kg/kg 作为烘干完成判定准则，模型计算得到烘干所需时间为{t_end_h:.4f}小时（即{t_end_s:.1f}秒）。烘
            干结束时刻中心含水率为{C_full[-1, 0]:.4f} kg/kg，表面含水率为{C_full[-1, -1]:.4f} kg/kg，全域
            最大含水率达到达标临界值 0.1500 kg/kg。表 5 及 result3.xlsx 均已导出。',
        }
    ]

```



```

        'evidence_required': ['model_results.json', 'metrics.json',
'table_index.json'],
        'evidence_status': 'computed',
    }
]

tables = [
    table_entry(
        question_id='AQ3',
        table_id='table5',
        title='药材烘干过程的水分浓度（表 5）',
        purpose='展示固定半径下每隔 6 小时及烘干结束时各截面含水率衰变历程。',
        path=t5_path,
        status='computed',
    ),
]

outputs = [
    {'path': str(t5_path), 'type': 'csv'},
    {'path': str(res3_path), 'type': 'xlsx'},
]

summary = f'AQ3 固定半径烘干时长确定完成：烘干达标时长为{t_end_h:.4f} h ({t_end_s:.1f} s)；
终点中心水分浓度为{C_full[-1, 0]:.4f} kg/kg, 表面为{C_full[-1, -1]:.4f} kg/kg。表 5 与
result3.xlsx 已导出。'

    upsert_question_contracts(
        question=QUESTION,
        result_summary=summary,
        metrics=metrics,
        tables=tables,
        conclusions=conclusions,
        outputs=outputs,
        status='computed',
        parameters=[{'name': 'N', 'value': N_run}, {'name': 'dt_s', 'value': dt_run},
{'name': 'event_status', 'value': diagnostics['status']}],
    )
    print(f'AQ3 finished successfully: t_end = {t_end_h:.4f} h ({t_end_s:.1f} s).')

if __name__ == '__main__':
    run_aq3()

```

## E.7 paper\_output/code/modeling/q4\_model.py

```

# 问题 4：使用题给数据求解并导出题目要求的结果。
from __future__ import annotations

import csv
import json
from pathlib import Path
import numpy as np
import pandas as pd
import openpyxl

from core_solver import simulate_fvm_moving, sample_and_interpolate_moving
from result_contract_io import (
    upsert_question_contracts,
    solver_metrics,
    write_csv_table,
    table_entry,
    round_float,
    PROJECT_ROOT,
    OUTPUT_DIR,
    TABLES_DIR,
    RESULTS_DIR,
    DATA_CLEANED_DIR,
)

QUESTION = {

```

```

        'question_id': 'AQ4',
        'title': '收缩域中的烘干时长',
        'task_type': '移动边界传热传质',
        'main_model': '物质坐标系(xi)移动网格 FVM + 附录 4 公式 + 离散阈值事件检测',
    }

def run_aq4():

config=json.loads((OUTPUT_DIR/'qa/model_validation_checks.json').read_text(encoding='utf-8
'))['selected_configuration']
N_run,dt_run=config['N'],config['dt_s']
env_path = DATA_CLEANED_DIR / 'a_environment.csv'
df_env = pd.read_csv(env_path)
t_env = df_env['time_s'].values.astype(np.float64)
T_env = df_env['temperature_K'].values.astype(np.float64)
C_env = df_env['air_moisture_kgkg'].values.astype(np.float64)

rad_path = DATA_CLEANED_DIR / 'a_radius.csv'
df_rad = pd.read_csv(rad_path)
t_rad = df_rad['time_s'].values.astype(np.float64)
R_rad = df_rad['radius_m'].values.astype(np.float64)

# 1. Run simulation until max(C) < 0.15 kg/kg (sampling every 60 s)
times_raw, radius_raw, T_raw, C_raw, end_time, n_rec, diagnostics =
simulate_fvm_moving(
    t_env=t_env,
    T_env=T_env,
    C_env=C_env,
    t_rad=t_rad,
    R_rad=R_rad,
    total_seconds=259200,
    sample_every_s=60,
    N=N_run,
    h=25.0,
    hm=8e-7,
    dt=dt_run,
    stop_at_cmax=0.15,
    return_diagnostics=True,
)
if diagnostics['status'] != 'reached':
    raise RuntimeError(diagnostics['status'])

t_end_s = end_time
t_end_h = t_end_s / 3600.0

# 2. Build Table 6 (Moisture every 6 h + drying end time)
# Target radii in domain: 0, 0.5, 1.0, 1.5 cm (and 2.0 cm if applicable, plus actual
surface)
r_tab_cm = np.array([0.0, 0.5, 1.0, 1.5, 2.0], dtype=np.float64)

max_6h = int(t_end_s // 21600)
times_6h_s = [i * 21600 for i in range(1, max_6h + 1)]
if t_end_s not in times_6h_s:
    times_tab_s = np.array(times_6h_s + [t_end_s], dtype=np.float64)
else:
    times_tab_s = np.array(times_6h_s, dtype=np.float64)

tab_indices = []
for ts in times_tab_s:
    idx = np.argmin(np.abs(times_raw - ts))
    tab_indices.append(idx)
tab_indices = np.array(tab_indices)

C_tab_interp, C_tab_surf = sample_and_interpolate_moving(
    times_tab_s,
    radius_raw[tab_indices],
    T_raw[tab_indices],
    C_raw[tab_indices],
    N=N_run,

```

```

        r_target_cm=r_tab_cm,
        t_env=t_env, T_env=T_env, C_env=C_env,
    )

    table6_rows = []
    for i, ts in enumerate(times_tab_s):
        if i == len(times_tab_s) - 1 and abs(ts - t_end_s) < 1e-3:
            time_label = f'烘干结束时间 ({t_end_h:.4f} h)'
        else:
            time_label = f'{ts / 3600.0:.1f}'
        row = {'时间/h': time_label}
        for j, r_val in enumerate(r_tab_cm):
            val = C_tab_interp[i, j]
            row[f'{r_val:g} cm'] = f'{val:.4f}' if not np.isnan(val) else ''
        row['药材表面'] = f'{C_tab_surf[i]:.4f}'
        row['当前半径/cm'] = f'{radius_raw[tab_indices[i]] * 100.0:.4f}'
        table6_rows.append(row)

    t6_path = TABLES_DIR / 'table_aq4_shrinkage_moisture.csv'
    write_csv_table(t6_path, table6_rows)

    # 3. Build result4.xlsx (every 60 s + end time, 0.1 cm steps + surface)
    r_full_cm = np.arange(21) / 10.0
    C_full_interp, C_full_surf = sample_and_interpolate_moving(
        times_raw,
        radius_raw,
        T_raw,
        C_raw,
        N=N_run,
        r_target_cm=r_full_cm,
        t_env=t_env, T_env=T_env, C_env=C_env,
    )

    res4_path = TABLES_DIR / 'result4.xlsx'
    wb = openpyxl.Workbook(write_only=True)
    ws = wb.create_sheet('Sheet1')

    header = ['时间\\到药材中心的距离'] + [round(r, 1) for r in r_full_cm] + ['药材表面']
    ws.append(header)

    for i in range(1, len(times_raw)):
        t_sec = float(times_raw[i])
        row = [t_sec]
        for val in C_full_interp[i]:
            row.append(round(val, 4) if not np.isnan(val) else None)
        row.append(round(C_full_surf[i], 4))
        ws.append(row)
    wb.save(res4_path)

    prior=json.loads((RESULTS_DIR/'metrics.json').read_text(encoding='utf-8'))
    t3=next(x['value'] for x in prior['items'] if x['metric_name']=='aq3_drying_time_s')
    reduction=(t3-t_end_s)/t3*100

    # 4. Metrics & Contracts
    metrics = [
        {'metric_name': 'aq4_drying_time_s', 'metric_role': 'evaluation', 'value':
        round_float(t_end_s, 2), 'unit': 's'},
        {'metric_name': 'aq4_drying_time_h', 'metric_role': 'evaluation', 'value':
        round_float(t_end_h, 4), 'unit': 'h'},
        {'metric_name': 'aq4_final_radius_cm', 'metric_role': 'evaluation', 'value':
        round_float(radius_raw[-1] * 100.0, 4), 'unit': 'cm'},
        {'metric_name': 'aq4_final_max_moisture', 'metric_role': 'evaluation', 'value':
        round_float(np.max(C_raw[-1]), 4), 'unit': 'kg/kg'},
        {'metric_name': 'aq4_final_min_moisture', 'metric_role': 'evaluation', 'value':
        round_float(np.min(C_raw[-1]), 4), 'unit': 'kg/kg'},
        {'metric_name': 'aq4_final_center_moisture', 'metric_role': 'evaluation', 'value':
        round_float(C_raw[-1, 0], 4), 'unit': 'kg/kg'},
        {'metric_name': 'aq4_final_surface_moisture', 'metric_role': 'evaluation',
        'value': round_float(C_full_surf[-1], 4), 'unit': 'kg/kg'},
        {'metric_name': 'aq4_time_reduction_vs_aq3_percent', 'metric_role': 'comparison',

```

```

'value': round_float(reduction, 2), 'unit': '%'},
]

metrics += solver_metrics(diagnostics,'AQ4')

conclusions = [
    {
        'question_id': 'AQ4',
        'conclusion_text': f'附件 2 收缩与附录 4 物性共同作用下，达标时长为{t_end_h:.4f} h
({t_end_s:.1f} s)，较问题三变化{-reduction:.4f}%。该差异是联合情景变化，不全部归因于收缩。终点
半径为{radius_raw[-1]*100:.4f} cm，表面含水率为{C_full_surf[-1]:.4f} kg/kg。',
        'evidence_required': ['model_results.json', 'metrics.json',
'table_index.json'],
        'evidence_status': 'computed',
    }
]

tables = [
    table_entry(
        question_id='AQ4',
        table_id='table6',
        title='药材烘干过程的水分浓度（表 6）',
        purpose='展示收域中每隔 6 小时及烘干结束时刻各物理截面及真实表面的含水率分布。',
        path=t6_path,
        status='computed',
    ),
]

outputs = [
    {'path': str(t6_path), 'type': 'csv'},
    {'path': str(res4_path), 'type': 'xlsx'},
]

summary = f'AQ4 收缩域烘干时长求解完成：烘干达标时长为{t_end_h:.4f} h ({t_end_s:.1f} s)；
点半径为{radius_raw[-1] * 100.0:.4f} cm，中心含水率为{C_raw[-1, 0]:.4f} kg/kg，表面为
{C_full_surf[-1]:.4f} kg/kg。表 6 与 result4.xlsx 已导出。'

upsert_question_contracts(
    question=QUESTION,
    result_summary=summary,
    metrics=metrics,
    tables=tables,
    conclusions=conclusions,
    outputs=outputs,
    status='computed',
    parameters=[{'name': 'N', 'value': N_run}, {'name': 'dt_s', 'value': dt_run},
{'name': 'event_status', 'value': diagnostics['status']}],
)
print(f'AQ4 finished successfully: t_end = {t_end_h:.4f} h ({t_end_s:.1f} s).')

if __name__ == '__main__':
    run_aq4()

```

## E.8 paper\_output/code/modeling/result\_contract\_io.py

```

"""四问共用的结果写出：CSV、指标、结论与运行来源。"""
from pathlib import Path
from datetime import datetime
import csv
import hashlib
import json
import sys

PROJECT_ROOT=Path(__file__).resolve().parents[3]
OUTPUT_DIR=PROJECT_ROOT/'paper_output'
RESULTS_DIR=OUTPUT_DIR/'results'
TABLES_DIR=OUTPUT_DIR/'tables'
DATA_CLEANED_DIR=OUTPUT_DIR/'data_cleaned'

```

```

def round_float(value, ndigits=6):
    return round(float(value), ndigits)

def rel(path):
    return Path(path).resolve().relative_to(PROJECT_ROOT).as_posix()

def write_csv_table(path, rows):
    with path.open('w', encoding='utf-8-sig', newline='') as f:
        writer = csv.DictWriter(f, fieldnames=list(rows[0]))
        writer.writeheader(); writer.writerows(rows)

def table_entry(question_id, table_id, title, purpose, path, status):
    return dict(question_id=question_id, table_id=table_id, title=title, purpose=purpose,
                path=rel(path), source='paper_output/code/modeling', status=status)

def solver_metrics(diag, qid):
    names = {'left_time_s': 's', 'end_time_s': 's', 'left_max_C': 'kg/kg', 'final_max_C': 'kg/kg',
            'water_balance': '1', 'step_water_balance': '1', 'residual_C': '1', 'residual_T': '1',
            'max_iterations': 'count', 'mean_iterations': 'count', 'steps': 'count',
            'center_excess_bound': 'kg/kg', 'status_code': '1'}
    return [dict(metric_name=qid.lower()
+ '_' + key, metric_role='validation', value=diag[key], unit=unit)
            for key, unit in names.items()]

def upsert_question_contracts(question, result_summary, metrics, tables, conclusions, outputs,
                             parameters=None, status='computed'):
    stamp = datetime.now().isoformat(timespec='seconds')
    qid = question['question_id']
    runner = Path(sys.argv[0]).resolve()
    artifacts = [rel(item['path']) for item in outputs]

    provenance = dict(source_code_path=rel(runner), source_code_sha256=hashlib.sha256(runner.read
+ _bytes()).hexdigest(),
                    helper_path=rel(Path(__file__)), run_command=f'{sys.executable} -B
{rel(runner)}', run_exit_code=0,
                    input_files=['paper_output/data_cleaned/a_environment.csv'] + (['paper_output/
data_cleaned/a_radius.csv'] if qid=='AQ4' else []),
                    output_artifacts=artifacts, generated_at=stamp)

    paths = [RESULTS_DIR/'model_results.json', RESULTS_DIR/'metrics.json', RESULTS_DIR/'conclusion
s.json', TABLES_DIR/'table_index.json']
    contracts = [json.loads(p.read_text(encoding='utf-8')) for p in paths]
    model, metric, conclusion, table = contracts

    item = dict(question_id=qid, title=question['title'], task_type=question['task_type'], result_t
ype='simulation',
                main_model=question['main_model'], baseline_model='', result_summary=result_summary,
                outputs=[dict(x, path=rel(x['path'])) for x in outputs], parameters=parameters or
[],
                evidence_status=status, status=status, execution_provenance=provenance)
    model['questions'] = [x for x in model['questions'] if x['question_id']!=qid] + [item]
    metric['items'] = [x for x in metric['items'] if x['question_id']!=qid] +
[dict(x, question_id=qid, status=status) for x in metrics]
    conclusion['items'] = [x for x in conclusion['items'] if x['question_id']!=qid]
+ conclusions
    ids = {x['table_id'] for x in tables}
    table['tables'] = [x for x in table['tables'] if x['table_id'] not in ids] + tables
    table['notes'] = ['各结果来自实际数值运行；显示四位小数不代表四位小数的物理准确度。']
    for path, data in zip(paths, contracts):
        data.update(schema_version='1.0', generated_by='paper_output/code/modeling/
result_contract_io.py', generated_at=stamp)
    path.write_text(json.dumps(data, ensure_ascii=False, indent=2, allow_nan=False)
+ '\n', encoding='utf-8')

```

## E.9 paper\_output/code/modeling/run\_modeling.py

```
"""依次运行四问、更新现有图表和真实运行记录。先执行数值验证脚本。"""
from pathlib import Path
from datetime import datetime, timezone
import hashlib
import json
import os
import subprocess
import sys
import platform

ROOT=Path(__file__).resolve().parents[3]
OUT=ROOT/'paper_output'
CODE=OUT/'code/modeling'

def record(path):
    return
dict(path=path.relative_to(ROOT).as_posix(),exists=True,bytes=path.stat().st_size,
    sha256=hashlib.sha256(path.read_bytes()).hexdigest())

def main():

validation=json.loads((OUT/'qa/model_validation_checks.json').read_text(encoding='utf-8'))
assert validation['status']=='PASS', '先完成数值验证'
for name,sha in validation['input_hashes'].items():
    assert hashlib.sha256((ROOT/name).read_bytes()).hexdigest()==sha, name
selected=[int(q) for q in sys.argv[1:]] if len(sys.argv)>1 else list(range(1,5))
runs=[]
if len(selected)<4:
    previous=json.loads((OUT/'results/run_manifest.json').read_text(encoding='utf-8'))
    runs=[r for r in previous['runs'] if not any(f'AQ{q}' in r['question_ids'] for q
in selected)]
    environment=dict(os.environ,PYTHONIOENCODING='utf-8',PYTHONDONTWRITEBYTECODE='1')
    for q in selected:
        script=CODE/f'q{q}_model.py'
        started=datetime.now(timezone.utc).isoformat()
        command=[sys.executable, '-B', str(script)]
        print(f'Running AQ{q}',flush=True)

result=subprocess.run(command,cwd=ROOT,env=environment,capture_output=True,text=True,encoding='utf-8')
print(result.stdout,flush=True)
if result.returncode:
    print(result.stderr,flush=True)
    raise RuntimeError(f'AQ{q} failed')
model=json.loads((OUT/'results/model_results.json').read_text(encoding='utf-8'))
item=next(x for x in model['questions'] if x['question_id']==f'AQ{q}')

inputs=[OUT/'data_cleaned/a_environment.csv',CODE/'core_solver.py',CODE/'result_contract_i
o.py',OUT/'qa/model_validation_checks.json']
if q==4: inputs.append(OUT/'data_cleaned/a_radius.csv')
runs.append(dict(run_id=f'AQ{q}_{started}',script=record(script)
['path'],script_sha256=record(script)['sha256'],
    question_ids=[f'AQ{q}'],command=f'{sys.executable} -B
{script.relative_to(ROOT).as_posix()}',returncode=0,status='PASS',

started_at=started,finished_at=datetime.now(timezone.utc).isoformat(),working_directory='.',

environment=dict(python=sys.version,platform=platform.platform()),input_files=[record(p)
for p in inputs],
    output_artifacts=[record(ROOT/p) for p in item['execution_provenance']]
['output_artifacts']],stdout_tail=result.stdout))
plot=OUT/'code/visualization/generate_model_evidence.py'
subprocess.run([sys.executable, '-B', str(plot)],cwd=ROOT,env=environment,check=True)
# 验证表和顺序对照保存在已有 JSON 中；正文直接引用这些真实计算记录。
```

```

metrics=json.loads((OUT/'results/metrics.json').read_text(encoding='utf-8'))
metrics['numerical_validation']=validation

(OUT/'results/metrics.json').write_text(json.dumps(metrics,ensure_ascii=False,indent=2,allow_nan=False)+'\n',encoding='utf-8')
index=json.loads((OUT/'tables/table_index.json').read_text(encoding='utf-8'))
additional=[('table7','AQ3','长程网格与时间步验证','accepted_combined_comparison'),
            ('table8','AQ4','三格顺序对照','sequential_contrast'),
            ('table9','AQ3','边界情景检验','scenarios')]
ids={x[0] for x in additional}
index['tables']=[x for x in index['tables'] if x['table_id'] not in ids]
for tid,qid,title,section in additional:

index['tables'].append(dict(table_id=tid,question_id=qid,title=title,purpose=title,

path='paper_output/qa/model_validation_checks.json',json_section=section,status='computed',
source='numerical_validation'))

(OUT/'tables/table_index.json').write_text(json.dumps(index,ensure_ascii=False,indent=2)+'\n',encoding='utf-8')
manifest=dict(schema_version='1.0',generated_by='paper_output/code/modeling/run_modeling.py',
              generated_at=datetime.now(timezone.utc).isoformat(),status='PASS',runs=runs,
              validation_provenance=validation['execution_provenance'],
              validation_record=record(OUT/'qa/model_validation_checks.json'),
              figure_generator=record(plot),
              final_artifacts=[record(OUT/p) for p in
['results/model_results.json','results/metrics.json','results/conclusions.json','tables/table_index.json','figure_index.json']],
              generator=record(Path(__file__)))

(OUT/'results/run_manifest.json').write_text(json.dumps(manifest,ensure_ascii=False,indent=2)+'\n',encoding='utf-8')
print('Four questions and figures complete.',flush=True)

if __name__=='__main__':
    main()

```

## E.10 paper\_output/code/visualization/generate\_model\_evidence.py

```

"""Generate reproducible AQ1-AQ4 figures."""
from __future__ import annotations

import csv
import hashlib
import json
import math
import re
from datetime import datetime, timezone
from pathlib import Path

import matplotlib.pyplot as plt

ROOT = Path(__file__).resolve().parents[3]
OUT = ROOT / "paper_output"
TABLES = OUT / "tables"
FIGURES = OUT / "figures"

def read_csv(name: str) -> tuple[list[str], list[list[float]]]:
    with (TABLES / name).open(encoding="utf-8-sig", newline="") as handle:
        rows = list(csv.reader(handle))
        header = rows[0]
        values = []
        for row in rows[1:]:
            parsed = []
            if row[0].startswith('烘干结束时间'):

```

```

        row[0]=re.search(r'([0-9.]+) h',row[0]).group(1)
    for value in row:
        try:
            parsed.append(float(value))
        except ValueError:
            parsed.append(math.nan)
    if parsed and math.isfinite(parsed[0]):
        values.append(parsed)
    return header, values

def sha256(path: Path) -> str:
    return hashlib.sha256(path.read_bytes()).hexdigest()

def save(fig: plt.Figure, name: str) -> Path:
    path = FIGURES / name
    fig.tight_layout()
    fig.savefig(path, dpi=160, bbox_inches="tight")
    plt.close(fig)
    return path

def main() -> None:
    FIGURES.mkdir(parents=True, exist_ok=True)
    _, q1t = read_csv("table_aq1_temperature.csv")
    _, q1m = read_csv("table_aq1_moisture.csv")
    q2t_h, q2t = read_csv("table_aq2_temperature.csv")
    _, q2m = read_csv("table_aq2_moisture.csv")
    _, q3 = read_csv("table_aq3_drying_moisture.csv")
    _, q4 = read_csv("table_aq4_shrinkage_moisture.csv")

    # AQ1: temporal evolution of centre/surface temperature and moisture.
    fig, axes = plt.subplots(1, 2, figsize=(10, 4))
    t = [row[0] / 60 for row in q1t]
    axes[0].plot(t, [row[1] for row in q1t], label="center")
    axes[0].plot(t, [row[-1] for row in q1t], label="surface")
    axes[0].set(xlabel="Time (min)", ylabel="Temperature (°C)", title="AQ1 temperature field")
    axes[1].plot([row[0] / 60 for row in q1m], [row[1] for row in q1m], label="center")
    axes[1].plot([row[0] / 60 for row in q1m], [row[-1] for row in q1m], label="surface")
    axes[1].set(xlabel="Time (min)", ylabel="Dry-basis moisture", title="AQ1 moisture field")
    for axis in axes:
        axis.legend(); axis.grid(alpha=0.25)
    p1 = save(fig, "fig_aq1_fields.png")

    # AQ2: radial profiles at each reported time.
    fig, axes = plt.subplots(1, 2, figsize=(10, 4))
    radius = [float(label.split())[0]) for label in q2t_h[1:]]
    for row in q2t:
        axes[0].plot(radius, row[1:], marker="o", label=f"{row[0]:g} h")
    for row in q2m:
        axes[1].plot(radius, row[1:], marker="o", label=f"{row[0]:g} h")
    axes[0].set(xlabel="Radius (cm)", ylabel="Temperature (°C)", title="AQ2 radial temperature")
    axes[1].set(xlabel="Radius (cm)", ylabel="Dry-basis moisture", title="AQ2 radial moisture")
    for axis in axes:
        axis.legend(fontsize=7); axis.grid(alpha=0.25)
    p2 = save(fig, "fig_aq2_profiles.png")

    # AQ3: whole-domain maximum versus threshold.
    fig, axis = plt.subplots(figsize=(6, 4))
    hours = [row[0] for row in q3]
    maxima = [max(value for value in row[1:] if math.isfinite(value)) for row in q3]
    axis.plot(hours, maxima, marker="o", label="domain maximum")
    axis.axhline(0.15, color="crimson", linestyle="--", label="threshold 0.15")
    axis.set(xlabel="Time (h)", ylabel="Dry-basis moisture", title="AQ3 drying threshold")
    axis.grid(alpha=0.25); axis.legend()

```



```

p3 = save(fig, "fig_aq3_threshold.png")

# AQ4: moving boundary and moisture profiles.
fig, axes = plt.subplots(1, 2, figsize=(10, 4))
hours = [row[0] for row in q4]
axes[0].plot(hours, [row[-1] for row in q4], marker="o", color="black")
axes[0].set(xlabel="Time (h)", ylabel="Radius (cm)", title="AQ4 shrinking domain")
for row in q4:
    values = [value for value in row[1:5] if math.isfinite(value)]
    axes[1].plot([.5*i for i in range(len(values))+[row[-1]], values+[row[-2]],
marker="o", label=f"{row[0]:g} h")
    axes[1].set(xlabel="Radius (cm)", ylabel="Dry-basis moisture", title="AQ4 moisture
profiles")
axes[1].legend(fontsize=7); axes[0].grid(alpha=0.25); axes[1].grid(alpha=0.25)
p4 = save(fig, "fig_aq4_shrinkage.png")

# Numerical validation is produced by run_numerical_validation.py; keep it separate
from plotting.
source_map = {
    "fig_aq1_fields": (p1, ["AQ1"], "table_aq1_temperature.csv"),
    "fig_aq2_profiles": (p2, ["AQ2"], "table_aq2_temperature.csv"),
    "fig_aq3_threshold": (p3, ["AQ3"], "table_aq3_drying_moisture.csv"),
    "fig_aq4_shrinkage": (p4, ["AQ4"], "table_aq4_shrinkage_moisture.csv"),
}
existing = json.loads((OUT / "figure_index.json").read_text(encoding="utf-8")) if (OUT
/ "figure_index.json").exists() else {}
index = {"schema_version": "1.0", "generated_at":
datetime.now(timezone.utc).isoformat(), "generated_by":
"paper_output/code/visualization/generate_model_evidence.py", "figures":
existing.get("figures", [])}
for fid, source, qids in (("fig_a_environment", "a_environment.csv", ["AQ1", "AQ2",
"AQ3", "AQ4"]), ("fig_a_radius", "a_radius.csv", ["AQ4"])):
    path = OUT / "figures" / f"{fid}.png"
    data = OUT / "data_cleaned" / source
    if path.exists() and data.exists() and not any(x.get("figure_id") == fid for x in
index["figures"]):
        index["figures"].append({"figure_id": fid, "title": fid, "path":
path.relative_to(ROOT).as_posix(), "expected_path": path.relative_to(ROOT).as_posix(),
"question_ids": qids, "source_data": data.relative_to(ROOT).as_posix(), "source_sha256":
sha256(data), "sha256": sha256(path), "bytes": path.stat().st_size, "status": "generated",
"ok": True, "exists": True, "placeholder": False, "evidence_type": "input_observation"})
    for figure_id, (path, qids, source) in source_map.items():
        item = {"figure_id": figure_id, "title": figure_id, "path":
path.relative_to(ROOT).as_posix(), "expected_path": path.relative_to(ROOT).as_posix(),
"question_ids": qids, "question_id": qids[0], "source_data":
f"paper_output/tables/{source}", "source_sha256": sha256(TABLES / source), "sha256":
sha256(path), "bytes": path.stat().st_size, "status": "generated", "ok": True, "exists":
True, "placeholder": False, "evidence_type": "model_result"}
        index["figures"] = [x for x in index["figures"] if x.get("figure_id") !=
figure_id] + [item]
    (OUT / "figure_index.json").write_text(json.dumps(index, ensure_ascii=False,
indent=2), encoding="utf-8")

if __name__ == "__main__":
    main()

```

## E.11 paper\_output/code/visualization/run\_numerical\_validation.py

```

"""数值检验与三格对照；只更新现有验证报告，不写临时轨迹文件。"""
from pathlib import Path
from datetime import datetime, timezone
import sys
import json
import hashlib
import time
import numpy as np
import pandas as pd

```

```

ROOT = Path(__file__).resolve().parents[3]
OUT = ROOT / 'paper_output'
sys.path.insert(0, str(OUT/'code/modeling'))
from core_solver import simulate_fvm_fixed, simulate_fvm_moving

def digest(path):
    return hashlib.sha256(path.read_bytes()).hexdigest()

def inputs():
    f = pd.read_csv(OUT/'data_cleaned/a_environment.csv')
    r = pd.read_csv(OUT/'data_cleaned/a_radius.csv')
    return
    (f.time_s.to_numpy(), f.temperature_K.to_numpy(), f.air_moisture_kgkg.to_numpy()),
    (r.time_s.to_numpy(), r.radius_m.to_numpy())

def
solve(env, rad, moving=False, mode=2, N=80, dt=1., converge=True, horizon=259200, threshold=.15):
    settings = dict(total_seconds=horizon, N=N, dt=dt, sample_every_s=60,
                    stop_at_cmax=threshold, return_diagnostics=True,
                    max_iterations=50 if converge else 2, converge=converge)
    if moving:
        z = simulate_fvm_moving(*env, *rad, **settings)
        times, R, T, C, end, count, diag = z
    else:
        z = simulate_fvm_fixed(*env, formula_mode=mode, **settings)
        times, T, C, end, count, diag = z
        R = np.full(len(times), .02)
    return dict(times=times, R=R, T=T, C=C, diagnostics=diag, N=N, dt=dt)

def compare(a, b):
    # 只比共同 60s 时刻 · 细网格插值到较粗网格单元中心。
    count = min(int(a['times'][-1]//60), int(b['times'][-1]//60))+1
    ra = (np.arange(a['N']+.5)/a['N'])
    rb = (np.arange(b['N']+.5)/b['N'])
    differences = {}
    for key in ['T', 'C']:
        err = max(np.max(np.abs(a[key][i]-np.interp(ra, rb, b[key][i])))) for i in
    range(count))
    differences['max_abs_'+key] = float(err)
    differences['event_relative_difference'] =
    float(abs(a['times'][-1]-b['times'][-1])/b['times'][-1])
    differences['pass'] = bool(differences['max_abs_T']<=.05 and
    differences['max_abs_C']<=5e-4
                                and differences['event_relative_difference']<=.005)
    return differences

def summarize(x):
    ids = sorted(set([0]+[i for i, t in enumerate(x['times']) if t in
    [1800, 10800, 21600, 86400, 172800]]+[len(x['times'])-1]))
    return dict(N=x['N'], dt_s=x['dt'], diagnostics=x['diagnostics'],
                checkpoints=[dict(time_s=float(x['times'][i]), radius_m=float(x['R'][i]),
                center_T=float(x['T'][i, 0]), surface_cell_T=float(x['T'][i, -1]),
                max_C=float(x['C'][i].max()), min_C=float(x['C'][i].min()),
                max_cell=int(np.argmax(x['C'][i])),
                center_reconstructed_C=float((9*x['C'][i, 0]-x['C'][i, 1])/8)) for i in
    ids])

def short_checks(env, rad):
    checks = {}
    for dt in [.5, 1., 2.]:
        z = simulate_fvm_fixed(*env, 10.3, 1, dt=dt, return_diagnostics=True)
        checks['sampling_'+str(dt)] = bool(np.allclose(z[0], np.r_[np.arange(11), 10.3]) and
        z[-1]['water_balance']<1e-6)
    uniform_env=(np.array([0., 60.]), np.array([301.15, 301.15]), np.array([2.55, 2.55]))

```

```

z=simulate_fvm_moving(*uniform_env,np.array([0.,60.]),np.array([.02,.019]),60,return_diagnostics=True)
checks['uniform_moving'] = bool(np.max(abs(z[2]-301.15))<1e-8 and np.max(abs(z[3]-2.55))<1e-8)
a=solve(env,rad,mode=3,horizon=3600,threshold=-1.)
b=solve(env,
(np.array([0.,3600.]),np.array([.02,.02])),moving=True,horizon=3600,threshold=-1.)
checks['constant_radius'] = bool(np.max(abs(a['T']-b['T']))<1e-8 and
np.max(abs(a['C']-b['C']))<1e-8)

z=simulate_fvm_fixed(*env,1800,1,sample_every_s=60,stop_at_cmax=2.549999,return_diagnostics=True)
checks['event_between_samples'] = bool(z[-1]['status']=='reached' and z[-1]
['left_max_C']>=2.549999 and z[-1]['final_max_C']<2.549999 and z[0][-1]==z[-1]
['end_time_s'])
z=simulate_fvm_fixed(*env,2,2,return_diagnostics=True,max_iterations=1)
checks['nonlinear_failure'] = z[-1]['status']=='solver_failed'
checks['not_reached'] = a['diagnostics']['status']=='not_reached'
return checks

def main():
    started=time.perf_counter()
    env,rad=inputs()
    report=dict(schema_version='2.0',generated_by='paper_output/code/visualization/
run_numerical_validation.py',

generated_at=datetime.now(timezone.utc).isoformat(),status='IN_PROGRESS',checks=short_checks(env,rad),runs={},comparisons={},scenarios={})
    path=OUT/'qa/model_validation_checks.json'
    def save():
        path.write_text(json.dumps(report,ensure_ascii=False,indent=2,allow_nan=False)
+'\n',encoding='utf-8')
    if not all(report['checks'].values()):
        report['status']='FAIL';save();raise RuntimeError(report['checks'])
    accepted={}
    fine_seeds={}
    for qid,moving in [('AQ3',False),('AQ4',True)]:
        batch={}
        for key,N,dt,converge in [('B0',80,1.,False),('B1',80,1.,True),('B2',160,1.,True),('B3',160,.5,True)]:
            tick=time.perf_counter()
            x=solve(env,rad,moving=moving,N=N,dt=dt,converge=converge)
            if x['diagnostics']['status']=='not_reached':
                x=solve(env,rad,moving=moving,N=N,dt=dt,converge=converge,horizon=604800)
            batch[key]=x
            item=summarize(x);item['elapsed_s']=time.perf_counter()-tick
            report['runs'][qid+'_'+key]=item
            print(qid,key,x['times'][-1],x['diagnostics']['status'],flush=True)
            save()
        report['comparisons'][qid]={label:compare(batch[a],batch[b]) for label,a,b in [
('iteration','B0','B1'),('space','B1','B2'),('time','B2','B3'),
('combined','B1','B3')]}
        accepted[qid]=batch['B1']
        fine_seeds[qid]=batch['B3']
        save()
    selected_N,selected_dt=80,1.
    baseline_ok=all(v[k]['pass'] for v in report['comparisons'].values() for k in
['space','time','combined'])
    report['refinement_comparisons']={}
    if not baseline_ok:
        coarse=fine_seeds
        N=160
        while True:
            fine={}
            comparisons={}
            for qid,moving in [('AQ3',False),('AQ4',True)]:
                fine[qid]=solve(env,rad,moving=moving,N=2*N,dt=.5)
                report['runs'][f'{qid}_N{2*N}_dt0.5']=summarize(fine[qid])

```

```

        comparisons[qid]=compare(coarse[qid],fine[qid])
        print('REFINE',qid,N,2*N,comparisons[qid],flush=True)
    report['refinement_comparisons'][str(N)]=comparisons
    save()
    if all(x['pass'] for x in comparisons.values()):
        temporal={}
        combined={}
        for qid,moving in [('AQ3',False),('AQ4',True)]:
            finer=solve(env,rad,moving=moving,N=2*N,dt=.25)
            report['runs'][f'{qid}_N{2*N}_dt0.25']=summarize(finer)
            temporal[qid]=compare(fine[qid],finer)
            combined[qid]=compare(coarse[qid],finer)
            print('REFINED_TIME',qid,temporal[qid],combined[qid],flush=True)
            report['accepted_time_comparison']=temporal
            report['accepted_combined_comparison']=combined
            if all(x['pass'] for x in temporal.values()) and all(x['pass'] for x in
combined.values()):
                accepted=coarse
                selected_N,selected_dt=N,.5
                break
    if N>=1280:
        report['status']='FAIL';save();raise RuntimeError('Refinement target not
reached')
        coarse=fine
        N*=2
        report['selected_configuration']=dict(N=selected_N,dt_s=selected_dt,max_iterations=50)
        save()
        configurations=[('appendix4_fixed',3,1.,False),('Ce_low',2,.8,False),
('Ce_high',2,1.2,False),('tail_hour',2,1.,True)]
        for name,mode,scale,tail in configurations:
            te,Ta,Ce=[v.copy() for v in env]
            Ce *= scale
            if tail:
                take=te>=10800
                meanT=np.trapezoid(Ta[take],te[take])/3600
                meanC=np.trapezoid(Ce[take],te[take])/3600
                te=np.r_[te,14400.+1e-6,604800.]
                Ta=np.r_[Ta,meanT,meanT];Ce=np.r_[Ce,meanC,meanC]
            x=solve((te,Ta,Ce),rad,mode=mode,N=selected_N,dt=selected_dt)
            if x['diagnostics']['status']=='not_reached':
                x=solve((te,Ta,Ce),rad,mode=mode,horizon=604800,N=selected_N,dt=selected_dt)
            report['scenarios'][name]=summarize(x)
            print(name,x['times'][-1],x['diagnostics']['status'],flush=True)
            save()
            t3=accepted['AQ3']['times'][-1];t4=accepted['AQ4']['times'][-1]
            tf=report['scenarios']['appendix4_fixed']['diagnostics']['end_time_s']
            report['sequential_contrast']=dict(t3F_s=float(t3),t4F_s=float(tf),t4S_s=float(t4),
property_change_percent=float((tf-t3)/t3*100),geometry_change_percent=float((t4-
tf)/t3*100),
total_change_percent=float((t4-t3)/t3*100),interpretation='指定先物性后几何路径；不估
计交互')
            for item in report['scenarios'].values():
                item['relative_time_change_percent']=(item['diagnostics']['end_time_s']-t3)/t3*100
            fields=[*report['runs'].values(),*report['scenarios'].values()]
            passed=all(v['diagnostics']['status']=='reached' and v['diagnostics']
['water_balance']<=1e-6 for v in fields)
            passed &= baseline_ok or (all(v['pass'] for v in
report['accepted_time_comparison'].values()) and all(v['pass'] for v in
report['accepted_combined_comparison'].values()))
            report['status']='PASS' if passed else 'FAIL'
            report['selected_configuration']=dict(N=selected_N,dt_s=selected_dt,max_iterations=50)
    if passed else None
        tracked=[Path(__file__),OUT/'code/modeling/core_solver.py',OUT/'data_cleaned/
a_environment.csv',OUT/'data_cleaned/a_radius.csv']
        report['input_hashes']={p.relative_to(ROOT).as_posix():digest(p) for p in tracked}
        report['elapsed_s']=time.perf_counter()-started
        report['execution_provenance']=dict(command=f'{sys.executable} -B
paper_output/code/visualization/run_numerical_validation.py',returncode=0 if passed else
1)
        save()

```

```

print(json.dumps({'status':report['status'],'comparisons':report['comparisons'],'sequential':report['sequential_contrast'],'elapsed_s':report['elapsed_s']},ensure_ascii=False),flush=True)
    return 0 if passed else 1

if __name__=='__main__':
    raise SystemExit(main())

```

## E.12 paper\_output/code/visualization/solver\_worker.py

```

"""单工况诊断入口：结果打印至标准输出，不创建临时文件。"""
import sys
import json
from run_numerical_validation import inputs,solve,summarize

if __name__=='__main__':
    kind,N,dt,horizon=sys.argv[1],int(sys.argv[2]),float(sys.argv[3]),int(sys.argv[4])
    env,rad=inputs()

    result=solve(env,rad,moving=kind.startswith('moving'),mode=int(kind[-1]),N=N,dt=dt,horizon=horizon)
    print(json.dumps(summarize(result),ensure_ascii=False,allow_nan=False))

```

## E.末 运行依赖

```

numpy>=2.0,<3
numba>=0.60,<1
pandas>=2.2,<4
matplotlib>=3.8,<4
seaborn>=0.13,<1
requests>=2.32,<3
python-docx>=1.1,<2
pypdf>=5,<7
openpyxl>=3.1,<4
xlrd>=2,<3
lxml>=5,<7
latex2mathml>=3.77,<4

```